



UNIVERSITÀ DEGLI STUDI DI MILANO

Facoltà di Scienze Politiche, Economiche e Sociali

Master's Degree Course in Data Science for Economics

AI-DRIVEN SATELLITE IMAGE SEGMENTATION FOR
ENHANCED FLIGHT SIMULATOR ENVIRONMENTS

Academic Year 2023-2024

Thesis by:

Giordano Vitale

Student ID: 14310A

Advisor: Professor Dario MALCHIODI

Co-advisor: Professor Nicolò CESA-BIANCHI

Company Advisors: Dr. Gianluca PARNISARI,
Dr. Raffaele BERZOINI

Abstract

Semantic segmentation has gained increasing attention and real-world applications since Deep Learning revolutionized the field. An example of such applications is the enhancement of a flight simulator’s visual features, aimed at making the virtual environment more realistic for its users.

To achieve this, a variety of state-of-the-art models, from different architectural families, are implemented, trained, and evaluated on a dataset consisting of satellite images. A substantial portion is dedicated to the review of such models, both the implemented and others not tested throughout this research. The best-performing model, determined by the Intersection over Union (IoU) metric, will be integrated into the production phase of the flight simulator software, offering more detailed and realistic visual scenes for users.

In the conclusive part of this work, potential improvements and overall considerations are presented and discussed, providing a forward-looking view of how the results can be developed in future implementations.

Contents

1	Introduction	1
1.1	Task Description & Motivation	1
1.2	Dataset Description	2
1.3	Outline	5
2	Background	6
2.1	Traditional Methods	6
2.2	Deep Learning Methods	8
2.3	Convolutional Neural Networks	12
2.3.1	Input Layer	13
2.3.2	Convolutional Layer	14
2.3.3	Activation Function	17
2.3.4	Pooling Layer	21
2.3.5	Batch Normalization	21
2.3.6	Dropout Layer	22
2.3.7	Transposed Convolutional Layer	23
2.3.8	Dense Layer	24
2.3.9	Output Layer	24
2.4	Implementation Details	25
2.4.1	Hardware and Software	25
2.4.2	Image Shape & Batch Size	25
2.4.3	Evaluation Metric	26
2.4.4	Loss Functions	27
2.4.5	Optimizer and Learning Rate	30
2.4.6	Class Weights	31
2.4.7	Augmentation Pipeline	32

3	Popular Segmentation Architectures	34
3.1	Fully Convolutional Networks	35
3.2	U-Net	36
3.3	SegNet	38
3.4	DeepLab	40
3.4.1	DeepLabV1	40
3.4.2	DeepLabV2	41
3.4.3	DeepLabV3	42
3.4.4	DeepLabV3+	43
3.5	Attention Mechanisms-based Architectures	44
3.5.1	Understanding Attention’s Functioning	45
3.5.2	Attention U-Net	49
4	Implemented Solutions	52
4.1	Baseline U-Net	53
4.1.1	Model Performance	54
4.2	Pre-trained Backbone U-Net	55
4.2.1	ResNet50 Backbone	56
4.2.2	EfficientNetB5 Backbone	58
4.3	DeepLabV3+	60
4.4	Attention U-Net	61
5	Results	63
5.1	Models Comparison	63
5.2	Out-of-Sample Predictions	66
6	Conclusions	69
6.1	Main Insights	69
6.2	Future Work	70
	Bibliography	71

List of Figures

1.1	Example of training image, mask pair	3
1.3	Examples of corrupted images.	4
1.2	Objects and Pixel Counts	4
2.1	Example of U-Net Architecture with visible contracting and expanding paths	11
2.2	Example of one image decomposition into three color channels.	13
2.3	Visual representation of a Convolutional Layer with kernel size=(3x3) and strides=(1,1). For simplicity, two easy pixels are involved and padding is not graphically included.	14
2.4	Example of one image feature extraction at different network's depths. The earlier layers capture broader patterns, whereas the deeper layers capture abstract patterns.	15
2.5	Strides=1 vs. Strides=2	16
2.6	ReLU graphical representation	18
2.7	Sigmoid function (black line) & tanh function (blue line).	20
2.8	MaxPooling vs. AveragePooling with 2x2 kernel	21
2.9	Dropout vs. Spatial Dropout Illustration	22
2.10	Transposed Convolutional Layer. In this visual example, the 4x4 input map is transposed to 7x7 output map.	23
2.11	IoU for class c Illustration	26
2.12	Augmentation steps applied to images only (top) and to both images and masks (bottom).	32
3.1	Example of encoder-decoder architecture with pre-trained backbone . . .	35
3.2	Fully Convolutional Network by Long et al. [39]	36
3.3	Original U-Net architecture by Ronneberger et al. [48]	37
3.4	SegNet Architecture with the corresponding layers, by Cipolla et al. [4]. .	39

3.5	Pooling operation according to SegNet. The left-hand side refers to the pooling during the encoder. The right-hand side refers to the feature upsampling during the decoder.	40
3.6	Dilated Convolution kernels with different rates (1,2,3)	41
3.7	Atrous Spatial Pyramid Pooling (ASPP) block, placed at the end of the encoder. The concatenated result is then upsampled to obtain the desired output shape.	42
3.8	Refined ASPP block. After the last step, colored in blue, upsampling is performed to obtain the desired output shape.	43
3.9	DeepLabV3+ complete architecture.	43
3.10	Visual Description of (Q,K,V) Vector Embeddings for each patch in the feature map. For Brevity, the first and the last only are reported.	45
3.11	Creation of the matrices Q,V,K from the single vector embeddings.	46
3.12	Visual Representation of the mathematical steps involving the matrices Q,K,V.	47
3.13	Attention Block in Attention U-Net	50
3.14	Attention Gate's Dynamics	51
4.1	Baseline U-Net Results	54
4.2	ResNet50-UNet Results	56
4.3	EfficientNet U-Net Results	59
4.4	DeepLabV3+ Results	60
4.5	Attention ResNet50 U-Net Results	62
5.1	From left to right: input images, ground truth masks, and predicted masks for each proposed model.	65
5.2	Out-of-sample Predictions	67

List of Tables

1.1	Datasets comparison according to highest IoU metric	5
2.1	Class Weights for class imbalance	31
4.1	Vanilla U-Net IoU scores, single-class and overall, on the validation set. .	55
4.2	ResNet50 U-Net IoU scores, single-class and overall, on the validation set.	57
4.3	EfficientNet U-Net IoU scores, single-class and overall, on the validation set.	59
4.4	DeepLabV3+ U-Net IoU scores, single-class and overall, on the validation set.	61
4.5	ResNet50 U-Net IoU scores, single-class and overall, on the validation set.	62
5.1	Overall Models Comparison of Validation IoU scores, single-class and overall, on the validation set, including the paper’s results.	63

1 Introduction

Semantic Segmentation is an increasingly popular discipline that has numerous fields of applications. It consists of classifying every pixel of an image, where a group of pixels represents a certain object (class). The final segmented image can be obtained by deploying supervised and/or unsupervised learning techniques. In the first case, the images forming the dataset are already labeled, i.e. there is a description for each pixel, defining which object that pixel is part of. In unsupervised learning, instead, there are no labels, and the learning process works differently. This thesis project revolves around supervised learning models and techniques.

Well known fields where image segmentation is widely adopted involve medical studies, where the goal is, for example, to define whether a cell is affected by an illness; environmental protection and disaster recovery, where it is deployed for the identification of objects within aerial images, also referred to as satellite images; precision agriculture, where this technology is helpful in predicting crop yields and to improve the farming performance.

In the context of aerial images, examples of pretty diverse objects the images are segmented into are roads, forest areas, roofs, and rivers. But the segmentation task can be applied to less-diverse objects such as pedestrian sidewalks, traffic lanes, white road lines, bicycle areas.

This chapter will introduce the reader to the general context this project is realized into. After describing the need of the current work (Section 1.1), an exhaustive description of the dataset is provided (Section 1.2), and lastly an outline of this report is shown (Section 1.3).

1.1 Task Description & Motivation

Satellite images segmentation is the core goal of this thesis project. Being a thesis developed alongside with TXT E-Tech, part of TXT Group, this project has a real-world-orientated approach. The above mentioned company, among many other services

and products, supplies other firms with engineering solutions in the aerospace industry, indeed one of the flagship products is an assortment of flight simulators for aircrafts.

The realism of such product, and hence its quality, heavily depends on the goodness of reproduction of the real.world environment environment. Naturally, a more precise representation of the landscapes, especially in the areas closer to the airports and heli-ports, results in a better product and consequently in a stronger position in the market against the competitors.

At the core of this research is the design of a Convolutional Neural Network (CNN) capable of distinguishing and categorizing each pixel of satellite images into specific classes, both from urban and rural environments. A granular segmentation is pivotal for generating precise 3D geometries that reflect real-world landscapes.

This research aims at contributing to the enhancement of the realism of flight simulator offered by TXT E-Tech, by leveraging AI-powered solutions based on the most recent Computer Vision algorithms.

1.2 Dataset Description

The dataset considered for this thesis project is *Land-cOVER Domain Adaptive* (Love-DA) [61]. This dataset consists of 5,987 high-spatial resolution images distributed across three parts: the training set covers 2,522 ($\sim 42\%$), the validation set 1,669 ($\sim 28\%$), and the test set the remaining 1,796 ($\sim 30\%$). In all sets, the image size corresponds to 1024x1024.

Each figure has a 0.3m spatial resolution, meaning that each pixel captures an area of 30x30 centimeters of land, allowing a deep detailed representation. The areas involved in these pictures are part of three Chinese cities, namely Nanjing, Changzhou, and Wuhan, covering more than 500km². Overall, nine urban areas and equivalent number of rural areas are selected to foster a good balance.

While the training and validation sets are annotated, the test set is unlabeled, meaning that there does not exist a mapping of each pixel to its corresponding class. For this reason, since this research is focusing on supervised learning techniques, the dataset considered for this thesis project only focuses on the training and validation sets, the latter being considered as the test set (1,669) for the experiments of models trained on the training set (2,522).

Each labeled image is equipped with a double annotation schema. One schema dis-

tinguishes the pixels of a certain image between "rural" and "urban" areas. The second schema, which is the one adopted in this work, discerns between the following seven classes: background, road, building, forest, water, agriculture, and barren. In other words, each RGB satellite image is paired with another corresponding image, called *mask*, which contains the ground truth labels for each pixel.

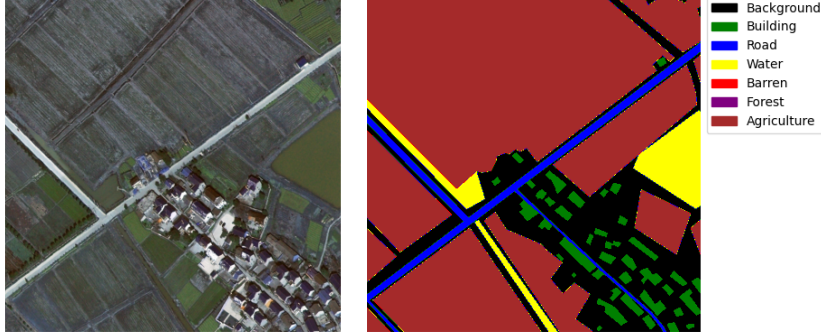


Figure 1.1: Example of training image, mask pair

By their own nature, the objects included in LoveDA have different scales. For instance, an agricultural land is, on average, severely larger than a building's roof. Similarly, a road seen from a satellite may look like an extremely thin stripe, when compared to a lake or, again, an agricultural area. Conversely, many buildings can be captured on a single image, because of their smaller size, especially if the image captures an urban landscape. Under the same logic, areas representing lakes (water) are wider and less numerous.

Moreover, an interesting information derived from the authors is the pixel distribution across the classes. This highlights how the background class dominates with respect to its counterparts. Fig. 1.2 taken from [61] illustrates these patterns.

The wide variety of objects and their physical characteristics, such as dimensions and shapes, hinders model generalization during the training phase, as the model faces images with inconsistent class distribution.

Following all the reasons recently described, addressing class unbalance becomes a crucial aspect to ensure the smaller, less represented objects do not get under-represented in favor of larger objects during model fitting and evaluation.

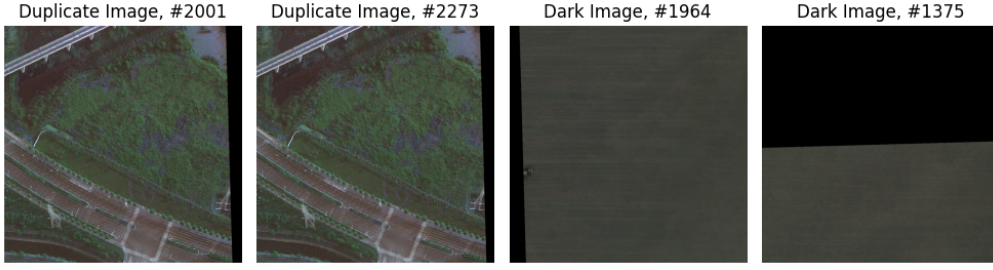
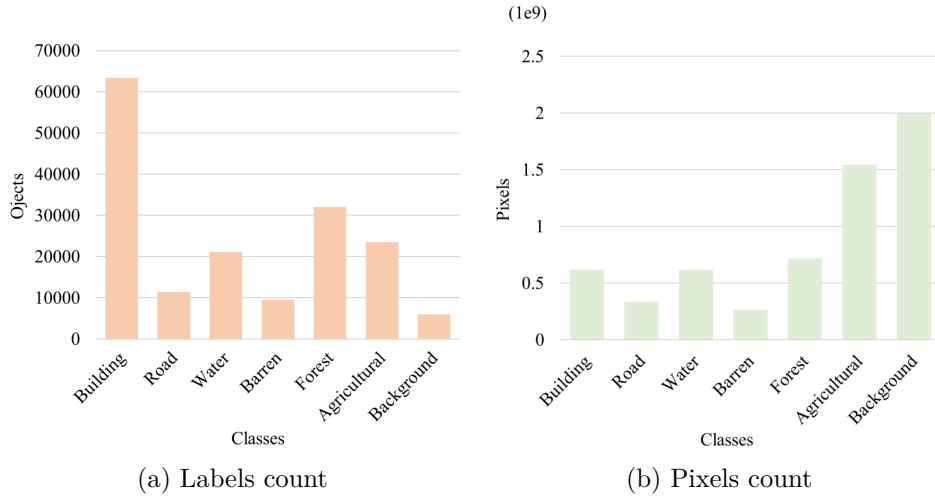


Figure 1.3: Examples of corrupted images.

Figure 1.2: Objects and Pixel Counts



The annotation process was carried out by professional remote sensing annotators using the ArcGIS geospatial software. After the first round of labeling, a human intervention has been performed to correct for wrong annotations, inaccurate boundaries and missing objects.

The dataset has been checked for potential poor instances, including exact or near duplicates, blurred, grayscale, too dark and/or too light images, utilizing the *CleanVision* library¹ in Python. As a result of such inspection, only one pair of training images were found to be duplicates, whereas a total of eleven images had very dark representations. These images, exemplified in Figure 1.3 have been removed from the training set.

When compared to similar datasets, LoveDA stands for its challengeability. In fact, as reported by [61] it is particularly complex to achieve high results, differently from the

¹<https://cleanvision.readthedocs.io>

other similar datasets shown in the Table (1.1). It is clear that the highest result in terms of Intersection over Union (IoU)² obtained for LoveDA is orders of magnitude lower than the achieved IoU metric for other, similar datasets utilized for semantic segmentation purposes. The results are obtained with respect to the best-performing model reported by the authors, i.e. HRNet.

Dataset	Top Mean IoU (%)
GID [59]	93.54
DeepGlobe [19]	74.10
ISPRS Potsdam	89.10
ISPRS Vaihingen	85.10
LoveDA	56.16

Table 1.1: Datasets comparison according to highest IoU metric

1.3 Outline

This thesis report provide a comprehensive overview about deep learning-based methods applied to Image Segmentation for satellite images. Chapter 2 introduces the necessary background, including past, modern, and future trajectories in Deep Learning and Image Segmentation. It also provides detailed instructions regarding the experimental setups. Chapter 3 describes the most popular models architectures that achieved successful results in segmentation tasks in a chronological fashion. Chapter 4 enumerates the implemented architectures, their results, and the corresponding considerations. Chapter 6 concludes the current work by analyzing the overall takeaways and insights, and by addressing future works and directions.

²Scores obtained from <https://paperswithcode.com/task/semantic-segmentation>.

2 Background

This chapter broadly covers the literature review, including past techniques, modern gold standard applications of Deep Learning as well as future trajectories in the field of Image Segmentation tasks. In the first section, a few traditional methods are closely listed and described. In the second section, Deep Learning is illustrated, including its main applications in the Computer Vision discipline. Section 2.3 deepens the perhaps most important Deep Learning tools in image data, i.e. Convolutional Neural Networks, and outlines their main components.

2.1 Traditional Methods

Before the advent of Deep Learning, Image Segmentation was a task performed using more traditional Computer Vision algorithms, including Thresholding, Clustering and Region-Based methods [38]. These techniques, which profoundly relied on handcrafted feature creation, required significant work, hence being strongly time-consuming. Moreover, they suffered from limited flexibility, generalization capabilities and robustness, especially when dealing with class imbalance [40].

Thresholding is usually applied to grayscale images, where each pixel carries a value between 0 and 255 along one single channel, and it outputs a binary mask. It is a technique which consists of replacing the value of a certain pixel with either 0, in case the intensity of that pixel is lower than a certain pre-determined threshold, or 1 otherwise. In its simplest setup, the threshold is selected by the expert who is performing Image Segmentation analysis, in which case the choice boils down to his intuition and knowledge, whereas in more complex setups it is determined by an algorithm. In the latter scenario, its value is determined so to maximize the similarity between the real partition, and the provided partition between the two classes [51].

Clustering analysis mainly operates in unsupervised learning setting and, when applied to images, it aims at grouping pixels that have more similar features, e.g. their color, intensity or location in the image, thus assigning similar pixels to the same class. The

most common method is K-means [17]. It starts by picking K centroids, then it assigns each pixel to its closest centroid dividing the image into K classes, and lastly averages all the pixel in the same cluster to update the centroids locations. The process is repeated until no significant update in the centers is observed.

Edge-based methods rely on the principle that the objects depicted in a given image often have clearly identifiable edges. For instance, the silhouette of a cat is deeply diverse from the silhouette of a tree. The main intuition behind this approach is that the pixels delimiting objects' boundaries record significant changes in the color intensity, hence they can be utilized as discriminators [3]. Once the edges are detected, the masks will be constructed as follows. There will be a value of 1 for the pixels corresponding to the detected edges, and 0 otherwise. The resulting shapes are then assigned a label, based on the detected boundaries. The most popular edge-based methods is perhaps the Sobel Operator, which deploys convolution filters to the original image both vertically and horizontally. The main disadvantages of such technique is that, should the same object be subject to many peaks in terms of color intensity, not necessarily happening along its own boundaries, this method becomes faulty, as it would generate False Positives edges. Moreover, in case the edge of a certain object is discontinuous, perhaps because it is intersected with the boundaries of another object, this technique is thus unable to capture complex patterns.

While edge-based methods focus on the outer parts of the objects populating an image, *Region-based methods* focus on the inner parts of the objects. The two family of methods can be considered complementary [32]. In fact, the main proposition is that the pixel belonging to the same object share common characteristics, and hence they can form an area, and be grouped in a set that will represent a certain object. Similarly to what mentioned above, when an object is not representable by a cohesive, homogeneous group of pixels, and hence the desired shared patterns are not observable, then region-based methods face severe limitations.

All the traditional segmentation methods fail in achieving accurate results when applied to complex images, because they rely on too simplistic assumptions about color patterns, intensities, and textures [45].

2.2 Deep Learning Methods

To overcome the above-mentioned limitations, Deep Learning techniques have been applied to image semantic segmentation and, since then, increasingly remarkable improvements have been recorded [24].

Deep Learning methods are popular for their ability to automatically extract and learn complex and abstract patterns from the data [18], such as images, by means of a sequence of trainable *layers* often organized in a hierarchical, modular setting, eventually crafting the so-called Artificial Neural Network (ANN) architecture.

A Neural Network is a computational model inspired by the structure and functioning of the human brain. It consists of layers of interconnected nodes, often referred to as neurons. The network is organized into three main groups of layers: the input layer, that receives the input data; the hidden layers, which are intermediate stages where the transformations and analysis occur; and the output layer, where predictions are output.

Given their ability to generalize feature learning, neural networks can be almost seamlessly applied to a variety of tasks and circumstances, without losing their predictive accuracy. From this perspective, Deep Learning models are significantly more versatile than traditional methods and even than Machine Learning applications.

Each layer is able to learn certain motifs of the fed input, and the novelty brought by Deep Learning is that what each layer learns is not meticulously designed by a human expert, but it is done by the Deep Learning model itself [37], through backpropagation algorithms.

Training a Neural Network

This means that training a neural network in supervised learning domain consists of feeding the network with labeled examples and the model tries to minimize the objective function (see Section 2.4.4) by iteratively adjusting its own internal parameters towards a minimum in the error measured by the loss function [36]. More specifically, the training phase can be described by the following steps:

- **Forward Propagation:** the first step consists of feeding the model with annotated examples in the input layer. For example, in case of semantic segmentation, the annotation is the corresponding mask; in case of image classification, the annotation is the label assigned to the image. Those inputs then follow the sequential

layers of the network. Each neuron in each layer performs a weighted sum of its input (that receives from the previous layers), adds a bias, applies an activation function, and delivers its outputs to the next layer, where the process is repeated. At the end of the forward propagation, when there are no further layers to stream the processed data to, the output layer performs predictions based on what the network has learned during the flow. In the first steps, the predictions are highly likely to be inaccurate, as the learning process requires to pass the data through the network several times before reaching a convergence to stable outcomes;

- **Loss function:** the loss function takes as input the predictions produced by the output layer and the true labels of the fed input, and compares them to quantify how close the predictions are with respect to the ground truth. It is the main element that provides a signal to the model, assessing whether it is following the right learning trajectory or not. Again, in the early stages, the loss function will probably have high values, and it will hopefully display a decreasing behavior as the network is trained;
- **Backpropagation:** once the loss has been computed, and hence the goodness of the predictions has been measured, the network needs to know how to adjust its internal parameters in order to improve and perform better on the next iterations. The core idea is to identify which weights and biases contributed the most to a bad prediction, and identifying those sources of errors can help the model improve on the next passes. The goal of backpropagation is to compute the gradient of the loss function with respect to each weight. The gradient tells the network how much and in which direction the weights need to be adjusted to reduce the error in future iterations to achieve better performances.
- **Gradient Descent:** after backpropagation, the network updates its weights using a method called Gradient Descent (Equation 2.1), that is an optimization algorithm that iteratively moves the weights in the direction that reduces the error the most.

The described sequence of elements is repeated for a certain number of iterations, often called epochs [27]. Over many epochs, the network gradually learns to produce more accurate predictions by tuning its weights. This means that the neural network receives the entire training dataset multiple times, i.e. the number of epochs.

In the context of training deep neural networks, especially those with many hidden layers, a serious challenge that often arises is the vanishing gradient problem. This phenomenon occurs when the gradients calculated during the backpropagation phase described above become extremely small and, as a result, the weight updates in the earlier layers become negligible [9]. This leads to a scenario in which the network produces poor performances, as it struggles to learn because of the ineffective weights updating.

Deep Learning methods were already acknowledged to be extremely powerful, but before recent times they struggled to demonstrate their potential. The biggest limitations that hindered their widespread adoption are strongly linked with hardware capabilities, computational resources, and availability of large volumes of data, all elements that only recently became accessible. In fact, crucial to the realization of state-of-the-art implementations is the availability of GPU computing resources [7], which allow for a significantly higher speed when compared to CPUs, and this hardware novelty certainly influenced the feasibility of such new, deeper models.

Deep Learning in Image Classification

The first important Deep Learning-related breakthrough in the Vision field that allowed to exploit these methods' potential, and hence apply them in real practical case, was registered in 2012, when Hinton et al. [34] published a pioneering Convolutional Neural Network architecture (see Section 2.3) named AlexNet that guaranteed them the best result in the ImageNet Contest, an image classification competition [20]. Their work includes many relevant innovations. The first concerned the *depth* of the proposed architecture: before their work, CNN architectures were shallower, whereas AlexNet consisted of 60M parameters and over 650,000 neurons. More in detail, their model layout consisted of five convolutional layers, part of them followed by pooling operations, and three fully-connected layers.

After that, many models architecture have been released, often enlarging the depth of the model, and they steadily improved their overall performances [55] up to reaching and surpassing human level performances in pattern recognition [28]. An example of such revolutionary models are ResNet architectures [29], which introduced the novelty of residual learning. Residual blocks not only allow to combine high-level feature maps with more general feature maps obtained at a previous stage along the convolutional network's layers, but also mark a significant contribution addressing the vanishing gradient problem

[29]. The main idea was that, given the increasingly deep structures of the proposed networks, it was hard and slow for the model to update its weights and biases during the backpropagation phase, exactly because of the models' depth. As a solution, the skip connections introduced between the model's layer were capable of allowing the gradient to back propagate faster, bypassing some layers, and, hence, to guide the model towards a better performance, avoiding the early plateau leading to unseen accuracy scores.

Deep Learning in Image Segmentation

While the above discussed breakthroughs are related to image classification tasks, where the goal is to assign a label to the entire image, Deep Learning is applied to a variety of other Computer Vision tasks, among which image segmentation. If the AlexNet model [34] can be considered a pioneering work for image classification, then the model introduced by Ronneberger et al. [48] can equivalently be considered as the main discovery in the image segmentation field back when it was published. Even though it was firstly introduced in the context of biomedical image segmentation, this pioneering architecture proved to be quite versatile, modular, and agile that it is widely adopted, and often improved or modified with additional elements, in many segmentation tasks, not only in medical applications. In fact, U-Net became a blueprint for other segmentation models. It is formed by an encoder-decoder architecture, which can be visualized as a U-like shape, from which derives the name, that combines downsampling through convolutional and pooling layers, and upsampling process to restore the original spatial dimensions. Skip connections are responsible for transferring fine grained details to the decoder part. A more in-depth analysis and description will be provided in Section 3.2.

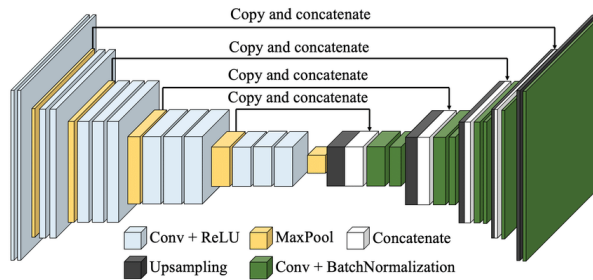


Figure 2.1: Example of U-Net Architecture with visible contracting and expanding paths

After the revolution brought by the U-Net model, many other successful architectures

have been elaborated for image segmentation. One of them certainly is the DeepLab family (including versions V1, V2, V3, V3+) [12, 14, 13, 15]. Their significant contributions were manifold, with each version of their architecture increasing the quality of the models, including the idea of combining deep Convolutional Neural Networks with Atrous Convolutions and Conditional Random Fields (CRFs) for refined edge detection, as well as the introduction of the so-called Atrous Spatial Pyramid Pooling (ASPP), which aimed at capturing the different scales of the Regions of Interest (ROI).

Further innovation was introduced by Attention-based mechanisms, firstly published by Vaswani et al. [60]. The core idea behind this model is to build an architecture capable of focusing only on the relevant portions of the images that convey significant, rich information that is worth learning from. Broadly speaking, this is performed through a weighting process where the deemed relevant portions of the image are assigned higher weights than the deemed less relevant ones, and the attention weights responsible for this task are learnable. Such method aims at reproducing the way humans focus on certain points of a certain object, as we use to focus on that object only often ignoring the background. The Attention mechanism can be used either on its own standalone structure for image segmentation, or it can be combined with a more classical Convolutional Neural Network architecture (see Section 3.5). An implementation of the second is proposed in Section 3.5.

Moreover, the Attention mechanism promoted the genesis of the currently most promising area of research, represented by Transformer-based models. While firstly introduced in the context of Natural Language Processing (NLP) [60], they can be applied to Computer Vision tasks, such as image segmentation indeed. Particularly important in translating NLP-Transformer logic into Computer Vision is the Vision Transformer (ViT) framework [21]. Transformers represent a paradigm shift from Convolutional Neural Network-based models to pure Transformer-based setups.

2.3 Convolutional Neural Networks

The most widespread application of Deep Learning in the field of Computer Vision is the family of Convolutional Neural Network (CNN) models. The previously mentioned AlexNet and ResNet architectures, for example, are part of this group. A Convolutional Neural Network is a particular type of neural network specifically utilized and designed for tasks like image processing [44].

Since Convolutional Neural Network belong to the broader family of neural networks, their architectures consists of one input layer, many hidden layers, and one output layer. Each of these sections is presented in the following pages.

2.3.1 Input Layer

The input layer is responsible for the import of the raw data (i.e. the input image) and translates it into a certain number of neurons, preparing it for the incoming procedure. In fact, when an image is processed by a Convolutional Neural Network, it undergoes several steps before it becomes usable by the network. An input image is typically in RGB (Red, Green, Blue) format, meaning that it has three channels, one for each primary color, and each channel is a 2D matrix of dimensions corresponding to the image's width and height. Each element of the matrix represents the intensity of the corresponding color at a specific pixel. For instance, if an image has dimensions of 256x256 pixels, each color channel will have a matrix of shape 256x256, resulting in 65,536 pixel values per channel. Given that there are three color channels, the total number of values representing the image amounts to $256 \times 256 \times 3 = 196,608$ values.

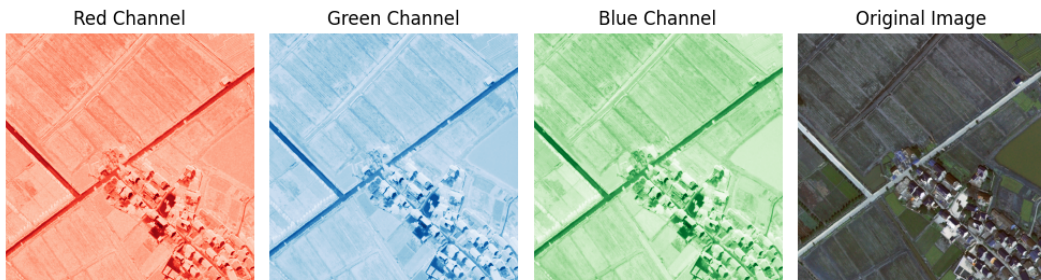


Figure 2.2: Example of one image decomposition into three color channels.

These pixel values are then fed into the input layer of the CNN, where each pixel value (or collection of values in the case of multiple channels) corresponds to a neuron. Alternatively, an RGB image of dimensions 256x256 pixels can be represented by a 3D tensor with the shape (256, 256, 3), where the last dimension corresponds to the three color channels. In practice, images (and masks for segmentation cases) are loaded and processed in groups, called batches, rather than individually. This is done to achieve more efficient data wrangling and GPU memory usage, as loading the entire training dataset at once is often unmanageable. Hence, the input tensor becomes a 4D tensor

with shape (batch size, height, width, channels). Continuing the example, if a batch size of 4 is chosen, the 4D tensor will have shape (4, 256, 256, 3). Before feeding the images into the network, they are typically scaled within an interval of values to ensure that the pixel values are in a range suitable for the model. This step is called Normalization and consists of scaling the pixel values. Common intervals are between 0 and 1, but also between -1 and 1 can be seen, depending on the specific preprocessing strategy employed by the selected model. As it will be discussed in Chapter 4, different models require different input scales.

From this point, the properly pre-processed image is ready to be fed into the subsequent hidden layers of the CNN that are responsible for the image's features extraction. In the following pages a number of common hidden layers is described.

2.3.2 Convolutional Layer

The convolutional layers are the central elements of a Convolutional Neural Network, as its main goal is to perform feature extraction by means of learnable kernels. In these layers a mathematical operation called convolution (Fig. 2.3) is applied over the image. This operation involves sliding a small matrix, known as a kernel, over the input image (both horizontally and vertically) to compute a weighted sum at each position. At each position occupied by the kernel on the image, the kernel performs an element-wise multiplication between its values and the corresponding values in the input image patch. The results are then summed into a single scalar value, which is placed at the corresponding location in the output feature map. The result is a set of feature maps that highlight specific characteristics of the image, such as edges, textures, spatial hierarchies, and many more abstract patterns.

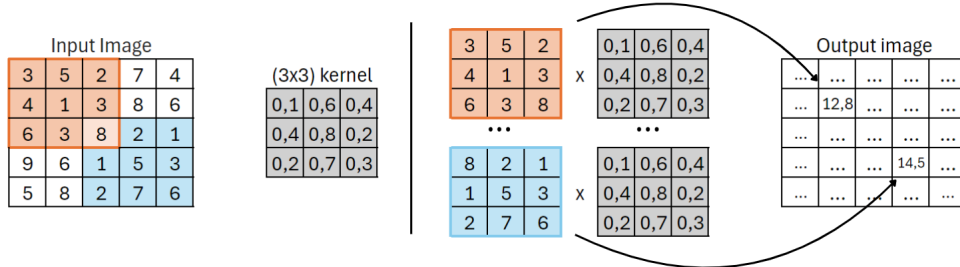


Figure 2.3: Visual representation of a Convolutional Layer with kernel size=(3x3) and strides=(1,1). For simplicity, two easy pixels are involved and padding is not graphically included.

A convolutional layer has many elements that can be customized. Each element governs the complexity of such operation and its resulting output.

One important element is the number of filters. Each filter learns a certain specific feature, and it determines the depth of the learned map that will be outputted. The consensus is that the learning dynamics follows a hierarchical structure: the first layers (fewer filters, e.g. 16,32) capture more broad and general motifs because the spatial transformations are still close to the raw data, whereas the deeper layers (numerous filters, e.g. 512, 1024) capture more abstract and intricate ones, due to the hierarchy of the network. An example of progressive feature extraction is provided in image 2.4.

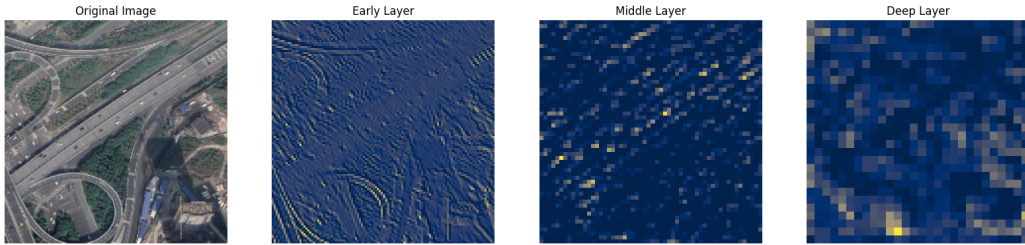


Figure 2.4: Example of one image feature extraction at different network's depths. The earlier layers capture broader patterns, whereas the deeper layers capture abstract patterns.

The kernel size defines the dimensions of the sliding window used during the convolution operation. It is typically a square $N \times N$ matrix, where N is a number significantly smaller than the height and width of the input image. The most common choice is to select a 3×3 kernel size, as it balances between capturing fine-grained details and maintaining computational efficiency, but also 5×5 or 7×7 filters are adopted. Larger kernels can capture broader patterns but at the expense of increased computation requirements. The values in the kernel matrix are called weights, and it is important to underline that each filter has its own kernel weights. Since each filter aims at capturing a particular motif of the input image, the direct implication is that, as the same kernel window slides over the image, the same kernel is able to detect precise motifs in different location of the image, thanks to its weights.

The kernel weights need to be initialized with certain values. The kernel initializer specifies the method used to initialize the weights of the convolutional filters. This step is crucial because the initial weights influence the learning process [35]. Most common initializers are random normal initialization, glorot uniform initialization [25], or He

initialization [28], each of which draws the weights values from a specific statistical distribution. After the backpropagation phase, these are the weights that are updated based on the gradient descent algorithm, and the updating property is the core of the principle that the kernels are learnable. The formula for updating the weights is typically:

$$w_{new} = w_{old} - \eta \cdot \nabla L, \quad (2.1)$$

where w_{old} is the previous weight, η is the learning rate (see 2.4.5), ∇L is the gradient of the loss with respect to the weights.

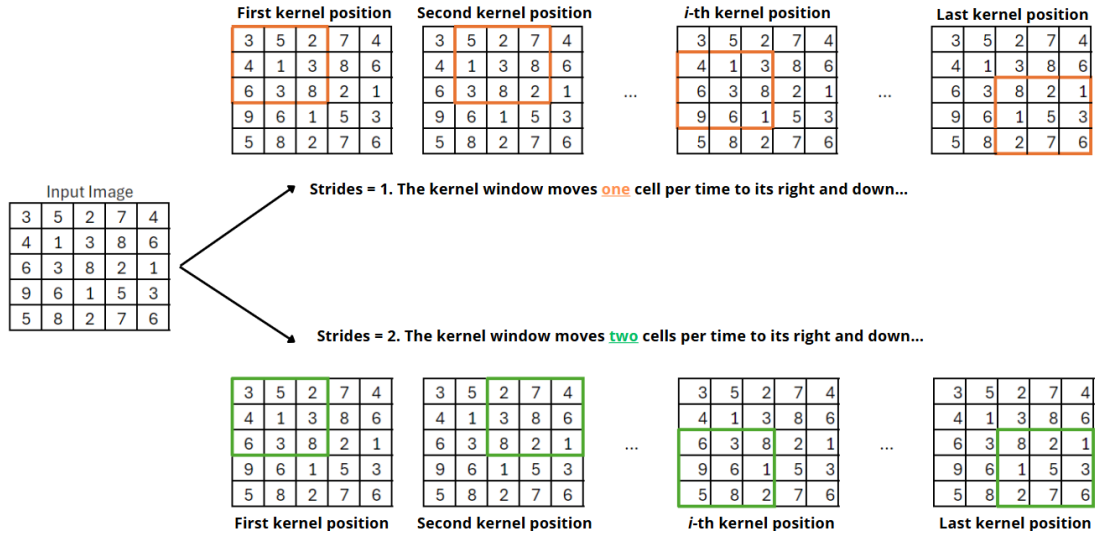


Figure 2.5: Strides=1 vs. Strides=2

The strides parameter indicates the step size by which the kernel moves across the input image. A stride of 1 means the filter moves one pixel at a time, and yields accurate feature maps at the cost of requiring more computational resources. Moreover, when a stride of 1 is selected, the spatial dimensions will not be affected. Conversely, a stride of 2 will make the kernel move two pixels at a time, and it will result in a coarser feature map, at the advantage of consuming fewer computational power. It produces an output map spatially smaller than the input. An illustration of these two step sizes is provided in Figure 2.5.

Another important parameter is padding, which determines how the input image is handled at its borders. Since the convolution operation may inherently reduce the

spatial dimensions of the input, information near the edges of the image may be lost. Padding helps mitigate this issue by adding additional pixels around the border of the image. Padding typically distinguishes between Valid and Same. Valid padding means no padding is applied, leading to smaller output feature maps. Same padding adds zeros around the borders, preserving the spatial dimensions of the input.

In general, the input of the convolutional operation is a tensor of shape $(\mathcal{B} \times \mathcal{H}_{in} \times \mathcal{W}_{in} \times \mathcal{C})$ where $\mathcal{B}$ is the batch size, $\mathcal{H}_{in}$ is the input height, $\mathcal{W}_{in}$ the input width, and $\mathcal{C}$ the number of channels. The output tensor from a convolutional layer has the shape $(\mathcal{B} \times \mathcal{H}_{out} \times \mathcal{W}_{out} \times \mathcal{F})$ where $\mathcal{B}$ is the batch size (equal to the input batch size), $\mathcal{H}_{out}$ is the output height, $\mathcal{W}_{out}$ the output width, and $\mathcal{F}$ is the number of filters used in the convolutional layer, which determines the depth of the output tensor feature map.

More in depth, the spatial dimensions of the output tensor, $\mathcal{H}_{out}$ and $\mathcal{W}_{out}$ can be vary depending on the kernel size, strides, and padding, provided the input dimensions. The output spatial dimensions are computed using Equations (2.2) and (2.3):

$$\mathcal{H}_{out} = \left\lfloor \frac{\mathcal{H}_{in} - K + 2P}{S} \right\rfloor + 1 \quad (2.2)$$

$$\mathcal{W}_{out} = \left\lfloor \frac{\mathcal{W}_{in} - K + 2P}{S} \right\rfloor + 1 \quad (2.3)$$

where K is the kernel size (e.g. 3x3 kernel size implies $K=3$), P is the padding policy (for example, 0 for valid padding, and for same padding it is computed to ensure the input dimensions are matched), S is the strides. In case one wants to maintain the output spatial dimensions equal to the input dimensions, then padding is applied when $S = 1$.

2.3.3 Activation Function

Each unit, or neuron, of each hidden layer receives as input the output of the previous layer, along with their corresponding weights and biases. Such input is passed to an activation function, that is crucial to determine which neuron-to-neuron connection to highlight, and which to suppress. This allows to govern which of the learned information to forward over the network, block after block until the end. The primary reason activation functions are so important is that they introduce non-linearity into the model [52].

Moreover, the choice of the activation function can significantly impact the vanishing gradient problem [26]. Without non-linearity, the entire network would behave like a single linear transformation and its learning ability would be strongly limited.

ReLU

One of the most utilized activation functions in the hidden layers is the Rectified Linear Unit (ReLU) [47]. The ReLU activation function is formally defined as:

$$\text{ReLU}(x) = \max(0, x), \quad (2.4)$$

where x is the input fed to the function. As a result, the output of such transformation is the input itself, x , in case it is positive, and 0 otherwise. For those units whose value is 0, they will not be updated anymore. Since ReLU outputs zero for negative inputs, it can introduce sparsity in the network, meaning that only some neurons are activated at any given time. This sparsity can help reduce overfitting and make the network more efficient. Unlike activation functions like sigmoid or tanh, ReLU does not saturate for large positive values. This helps mitigate the vanishing gradient problem [2].

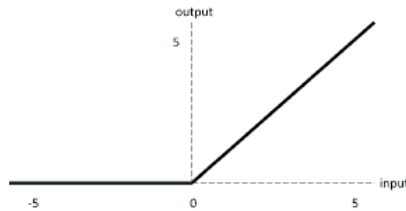


Figure 2.6: ReLU graphical representation

However, ReLU is not always the gold standard. In fact, if a large number of units output a negative value, the entirety of such neurons will be muted. As a consequence the network will likely underperform.

Leaky ReLU

One alternative activation function certainly is the Leaky ReLU [41]. This is a modified ReLU which has a positive slope in the negative values of the input as well, though

the inclination is usually lower. It is mathematically defined as:

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (2.5)$$

where α is a small positive constant that aims at avoiding the dying neurons phenomenon previously mentioned. Such small positive number allows neurons to have small gradient values, and thus prevents them from being muted.

PReLU

Another variant of ReLU is the Parametric Rectified Linear Unit (PReLU) [28]. The main difference with respect to Leaky ReLU is that now the parameter α becomes learnable, instead of being a fixed constant. This novelty introduces more flexibility at the cost of estimating an additional parameter, which will increase the computational overload. The mathematical formula of PReLU is the same as in Equation 2.5.

Sigmoid and Tanh

While the three mentioned activation functions are nowadays widely adopted, there exist many other functions, included the Sigmoid and the Tanh activation functions. The Sigmoid forces the input into the interval $[0,1]$. This is a suitable characteristic when working with binary classification tasks, as mapping the inputs into the range $[0,1]$ in the last, final output layer allows to interpret them as probabilities.

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (2.6)$$

Throughout the current research, Sigmoid has not been utilized, as the focus is multi-class pixel-wise classification. The Tanh (Hyperbolic Tangent) is comparable to the Sigmoid, except that the range within which the input is mapped differs, namely $[-1,1]$. When compared to Sigmoid, in certain cases the Tanh can yield more accurate results as the outputs are centered on 0.

$$\text{Tanh}(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (2.7)$$

Both the Sigmoid and the Tanh, however, suffer from the vanishing gradient problem

(see *Training a Neural Network* in Section 2.2), because when a large positive or large negative number is fed into the function, the gradient becomes close to zero for very large or small inputs. Take, for example, the Sigmoid function. When very large (positive or negative) values are passed to it, the function returns a value which is extremely close to 0 or 1, which are the extremes of the function's range. In either case, when this happens, the slope of the Sigmoid curve (nothing but the gradient!) in those farthest regions becomes zero.

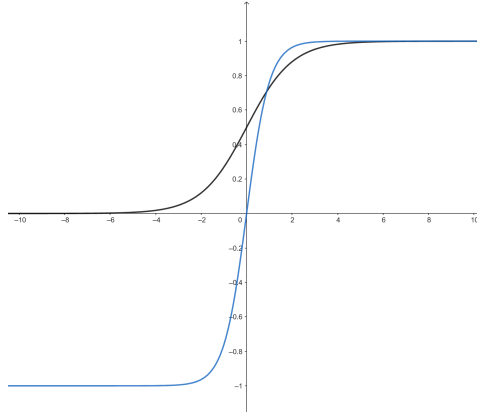


Figure 2.7: Sigmoid function (black line) & tanh function (blue line).

For the Tanh the situation is similar, with the only exception of the lower bound, though the flattening behavior is the same. As a result, during backpropagation, the gradients flowing through the network become negligible, causing the learning process to stagnate.

Softmax

The last activation function to be introduced is the Softmax activation function, which ensures that the output values are in the range $[0, 1]$ and their sum is equal to 1, hence it allows to interpret the final transformed results as probabilities. From this perspective, Softmax in multi-class applications can be compared to the role of Sigmoid in binary classification cases.

The Softmax function for the i -th class is defined as

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}, \quad (2.8)$$

where z represents the raw output of the network that needs to be softmaxed, z_i is its i -th element and hence represents the i -th class, N is the number of classes.

2.3.4 Pooling Layer

The pooling layer reduces the dimensionality of the learned feature map, i.e. the output coming from the convolutional layer after being passed to the activation function. It works as a "summary" of what has been learned in the previous convolutional operation [66]. In fact, its main goal is to down-sample the spatial dimensions of the feature maps: in other words, it lowers the computational burden of the network and helps retaining the essential information. By shrinking the data that will be processed in the subsequent layers it also works as a contributor to avoid overfitting, because it aims at retaining only the core information [44].

Two main Pooling operations often used are Max Pooling and Average Pooling. Both have two parameters: the kernel size k , already mentioned in the convolutional layer, that is the size of the window through which the pooling operation is performed, and the stride s , which determines by the number of steps by which the kernel size k will move horizontally and vertically across the input feature map at each step. In MaxPooling, the kernel window of size $(k \times k)$ slides across the input feature map and outputs the highest seen value within that region, and it repeats this procedure for each image patch taken under inspection. In AveragePooling the dynamics are similar, but instead of selecting the maximum value within the selected region, it returns the average of all the values in that window.

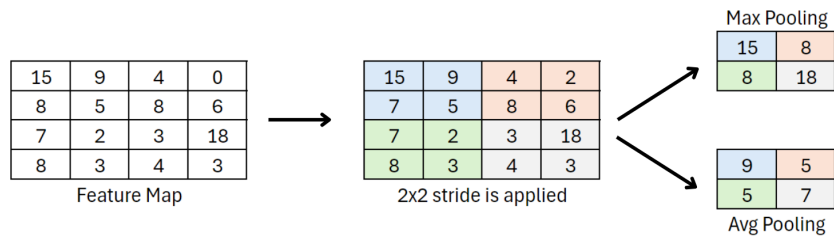


Figure 2.8: MaxPooling vs. AveragePooling with 2x2 kernel

2.3.5 Batch Normalization

Batch Normalization is usually applied between the convolution operation and the activation function and it consists of normalizing the output tensor from the previous

convolution, meaning that the tensor's values are transformed to have a mean of 0 and a standard deviation of 1. The main purpose of batch normalization is that it reduces the internal covariate shift [30], i.e. the situation in which the distribution of the inputs changes. When such phenomenon occurs, the learning process suffers from a deceleration, because the layers have to adapt to the new, different input distribution. For this reason, batch normalization allows the model to adapt faster to the different input scales, and as a consequence it accelerates convergence and model robustness [8, 50]. Despite being optional, meaning that a CNN architecture functions even without the support of BN, it has a good reputation and it has been empirically demonstrated that it is associated with superior performances [30].

2.3.6 Dropout Layer

Dropout is a regularization technique that addresses overfitting by randomly *muting*, given a certain probability parameter p , the neurons of a given hidden layer during the forward pass, to avoid the network to learn features that are overly specific of the input but not generalizable [5]. For example, if the parameter $p = 0.3$, then 30% of the neurons of that layer will be set to 0, and the information they carried will be aborted in the forward pass. Many experts, among which Ioffe et al. [30, 23], believe Dropout and Batch Normalization may be seen as alternatives and should not be applied together, because it could result in excessive regularization, but other believe that the two can go together as they can be interpreted as complementary [11].

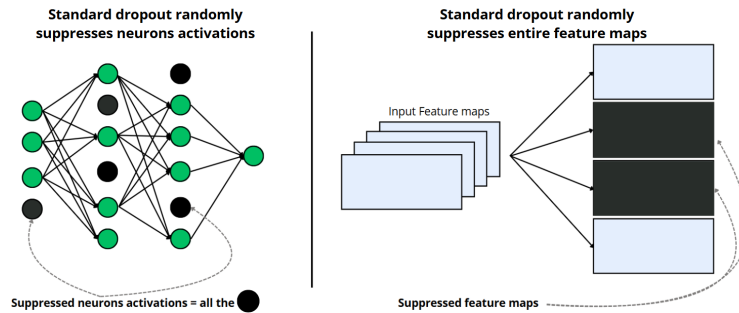


Figure 2.9: Dropout vs. Spatial Dropout Illustration

In addition to the standard Dropout layer, Spatial Dropout 2D is a regularization technique that has been applied in some of the experimented architectures during this thesis project. This kind of dropout is particularly well-suited for neural networks that

are made of convolutional layers. In a convolutional neural network, feature maps often exhibit strong spatial correlations, in other words neighboring pixels in an image contain related information. Standard Dropout may disrupt these correlations by dropping individual neurons, and this would potentially waste useful information that can ultimately impact the model's performance. Spatial Dropout 2D addresses this limitation of standard Dropout by randomly dropping entire feature maps instead of individual neurons within a feature map, promoting preservation of spatial coherence [58]. A graphical comparison of these two alternative methods is depicted in Figure 2.9.

2.3.7 Transposed Convolutional Layer

In the previous sections, it has been discussed how Convolutional and Pooling Layers are deployed handle the down-sizing of the spatial dimensions (height and width) of the input image. In semantic segmentation, after having obtained the output feature maps from the down-sampling sequence of layers, it is desirable to restore the initial spatial dimensions, to allow the learned patterns to be applied pixel-wise. To achieve this upsampling, the Transposed Convolutional Layer is applied. Similarly to its counterpart, this layer has parameter such as Strides and Padding, and they have the same functioning described in Convolutional Layers.

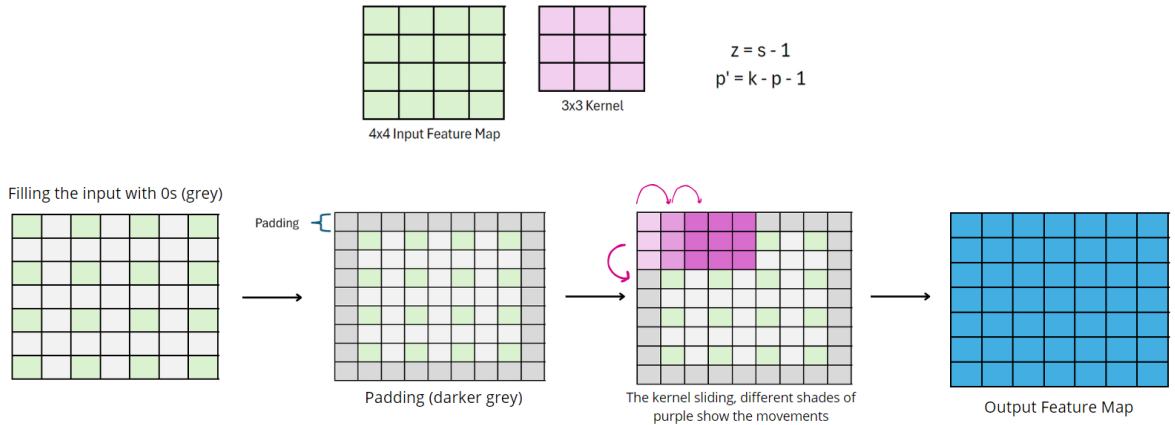


Figure 2.10: Transposed Convolutional Layer. In this visual example, the 4x4 input map is transposed to 7x7 output map.

Given the input feature map of dimension $H_{in} \times W_{in}$, the kernel window of size $k \times k$, the padding p and the strides s , the operation is computed as follows. First, more

parameters, namely z and p' , are obtained via $z = s - 1$ and $p' = k - p - 1$. The value of z determines how many zeros to insert between the input feature map's rows and columns, which is the first step, whereas the parameter p' will determine how many zeros will be added at the borders of the input map, after that it has been filled with zeros, completing the second step. Once the feature map has been expanded and padded, the third step consists of making the kernel slide over the processed feature map, and it will move according to the strides parameter. At this point, an element-wise multiplication and summation is performed and the output is obtained. The result of this operation is an upsampled output feature map where the spatial dimensions $H_{out} \times W_{out}$ are larger than the input dimensions $H_{in} \times W_{in}$. Figure 2.10 shows the described operation. The output height and width of the transposed convolution can be computed as:

$$H_{out} = s \cdot (H_{in} - 1) + k - 2p, \quad W_{out} = s \cdot (W_{in} - 1) + k - 2p \quad (2.9)$$

where the value of p is selected to ensure the output size matches the desired shape.

2.3.8 Dense Layer

Dense layers are particularly used in neural networks applied to classification tasks. In a dense layer, every unit is connected with all the previous layer's units, and for this characteristic they are also known as fully connected layers. The output coming from the previous step is first flattened, thus making it 1D, and then it passed to the Dense layer. As previously mentioned, this kind of layer is particularly useful in the final stage of an image classification task, because it allows the learned maps to be translated into a specific set of output categories, namely the classes object of the prediction.

For tasks like image semantic segmentation, Dense layers are not utilized, because they do not maintain spatial information due to the flattening preparation before the Dense layer itself, which is the exact opposite of what semantic segmentation aims at. When a 2D image or 3D tensor is flattened, all the spatial proportion are lost.

2.3.9 Output Layer

Now that the inner part has been described, the focus can shift to the last, output part of a Convolutional Neural Network. The output layer is where the learned relationships are used to express (compute) a prediction or another result, depending on the task the

model is applied to. In case of binary image classification, for example, there will be one single output neuron, which determines the probability of the image belonging to the class tagged with "1", whereas the complementary value will express the probability of that image belongs to the other class, 0. Such operation is handled by the Sigmoid function, as previously introduced.

In case of image segmentation, the focus of this thesis, the output layer requires a preparatory discussion on segmentation architectures, which differ significantly from the ones deployed in other Computer Vision tasks such as image classification. Since this paragraph is dedicated to Convolutional Neural Networks in general, a proper introduction will be provided in the coming sections (see Chapter 3). Without examining further, in image segmentation tasks the output layer returns a probability distribution over the classes for each pixel, meaning that each pixel will have a probability p_i of belonging to the i -th class for any of the existing classes, and the class with the highest probability will be the predicted label assigned to that pixel. This projection of the learned parameters into final results such as probability distribution is made possible by the Softmax activation function in multi-class classification tasks.

2.4 Implementation Details

2.4.1 Hardware and Software

All the models are written using the programming language Python3.10 under the TensorFlow/Keras framework, and they have been trained on a NVIDIA RTX A1000 6GB GPU for 100 epochs. However, an Early Stopping policy has been included to balance the amount of time required for a training run and the necessity of respecting the time constraint.

2.4.2 Image Shape & Batch Size

Due to the limited GPU memory, all the images and masks passed to the models are of dimension 256x256, undergoing a significant under-scaling from the original 1024x1024, and loaded in a batch size of 4. Once the definitive experiments were conducted, the input was increased to 512x512.

2.4.3 Evaluation Metric

The Mean Intersection over Union (IoU), considered as the gold standard in these applications [46], is the selected evaluation metric, both for an overall and single-class assessment.

The IoU computes the similarity between the predicted and the ground truth masks, for a given class (Figure 2.11). Provided a ground truth mask and the corresponding prediction, the IoU for a specific class computes the ratio between the number of overlapping pixels (i.e. the intersection) and the total number of pixels present in either the predicted or ground truth mask (i.e. the union), for that given class:

$$\text{IoU}_c = \frac{|\text{Prediction}_c \cap \text{Ground Truth}_c|}{|\text{Prediction}_c \cup \text{Ground Truth}_c|} = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c},$$

where TP are True Positives, FP are False Positives, and FN are False Negatives, all with respect to the class c . After calculating the IoU for each single class, the Mean IoU can be computed with a simple average across the classes:

$$\text{Mean IoU} = \frac{1}{N} \sum_{c=1}^N \text{IoU}_c, \quad (2.10)$$

with N being the number of classes.

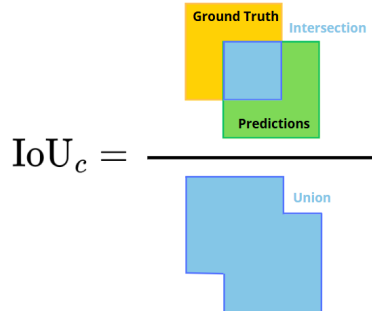


Figure 2.11: IoU for class c Illustration

Differently from other standard metrics such as the Accuracy score, the IoU effectively takes into account of the class imbalance issue usually present in such a problem setting [62]. The Accuracy score is not reliable in image segmentation tasks. By definition it counts the number of correct classifications over the number of instances to be classified

in general, but such measurement can be heavily skewed by the presence of one dominant class, such as the Background.

2.4.4 Loss Functions

During the experimentation phase several models' configurations have been tested, and three loss functions were examined: (i) the Focal Tversky Loss (FTL), (ii) a loss function originally proposed by Wang et al. [61], (iii) and a custom loss that combined the first two. All the chosen loss function have the goal to penalize harshly bad predictions, in order to strongly signal the model the right direction it has to proceed towards. They also are designed to address class imbalance. All the chosen loss functions make use of class weights. The selected weights are introduced immediately after the current discussion and can be read in Table 2.1.

Focal Tversky Loss

The first chosen objective function is the Weighted Tversky Loss (WTL), which is well known to be effective in imbalanced datasets [31, 49]. It is based on the Tversky Index (TI), that is a similarity measure, i.e. it is responsible for mathematically quantifying how much two sets are similar. In the context of image semantic segmentation, the similarity is computed with respect to the true labels and the predicted ones with the goal to assess the prediction accuracy.

The general Tversky Index formula is the following:

$$TI = \frac{TP}{TP + \alpha FN + \beta FP}, \quad (2.11)$$

where α calibrates the significance of False Negatives, while (β) calibrates the weight of False Positives predictions. Provided $\alpha + \beta = 1$, if $\alpha > \beta$, then False Negatives are penalized more than False Positives. This choice is adopted when the model predicts False Negatives more often, hence it requires guidance on what to penalize more harshly.

To the extent of this project, the Tversky Index in Equation (2.11) is computed with respect to each single label. These quantities are then utilized to obtain the weighted Tversky Index (2.12) as follows:

$$\text{Weighted TI} = \frac{\sum_{c=1}^7 \text{ClassWeight}_c \cdot \text{TI}_c}{\text{Weights Sum}} \quad (2.12)$$

The final Focal Tversky loss selected in the training phase is lastly obtained by:

$$L_1 = (1 - \text{Weighted TI})^\gamma, \quad (2.13)$$

where γ is an hyperparameter that defines the importance of hard-to-classify examples against the importance of the easier classification examples. More in detail, setting a value of $\gamma > 1$ magnifies the punishment for misclassifying a complex example, hence it helps instructing the model to pay more attention to the hard examples. Throughout the experiments of this project, $\gamma = \frac{4}{3}$ following the evidence from [1].

The values of α and β have been established for each individual class' Tversky Index based on the evidence gathered during the experiments conducted. In case of Background class, for example, despite the weighting policy described in 2.4.6, the number of False Positives was systematically higher than the number of False Negatives, and as a consequence the value of α was lower than the value of β . A similar reasoning has been applied to all the other classes, and when there was no significant asymmetry between False Positives and False Negatives, α was set equal to β at 0.5.

Loss Function as per *LoveDA*

The second loss function is a combination of more loss functions which has the idea to capture both the overall predictions accuracy and the single-class accuracy computed with respect to the Background class, as it is, once again, a potential source of class imbalance. The second loss function is defined as follows:

$$L_2 = L_{cce} + \alpha \cdot L_{bce} + \beta \cdot L_{dice}, \quad (2.14)$$

where L_{cce} is the categorical cross-entropy over all the classes, L_{bce} is the binary cross-entropy computed with respect to the background class, and L_{dice} is the Dice loss over all the classes. The values of α and β have been chosen at 0.5, following the evidence obtained by [61].

The loss values obtained from the component L_{cce} are in a vector of length 7, i.e. the

number of classes. To obtain a single scalar value from this function, the obtained values are averaged. The obtained value is then summed to the component L_{bce} and to the component L_{dice} , which are both a scalar value already.

More precisely, the Dice loss is computed in a similar fashion to the FTL. The difference between the Dice score and Equation (2.11) is that the True Positives are multiplied by a factor of 2, both at the numerator and at the denominator:

$$\text{Dice Score} = \frac{2 \cdot TP}{2 \cdot TP + \alpha FN + \beta FP}, \quad (2.15)$$

where α and β have the same scope as the one described a few lines above.

To preserve a homogeneous and fair approach, the Dice Score is computed for each single class and then it weighted by the class weights distribution, coherently following the approach described for the Focal Tversky Loss:

$$\text{Weighted Dice Score} = \frac{\sum_{c=1}^7 \text{ClassWeight}_c \cdot \text{Dice Score}_c}{\text{Weights Sum}} \quad (2.16)$$

The Dice loss is finally obtained as:

$$L_{dice} = 1 - \text{Weighted Dice Score} \quad (2.17)$$

Custom Loss Function

A third loss function has been engineered in order to craft a solution that could improve the models performances. It consists of a combination of the losses presented in this paragraph. While the loss function in Equation (2.17) utilizes the Dice score in its elements, the following proposed customized loss

$$L_3 = L_{cce} + \alpha \cdot L_{bce} + \beta \cdot L_{FTL}, \quad (2.18)$$

replaces it with the above mentioned Focal Tversky Loss, with the goal to capture more precisely False Negatives and False positives arising from class imbalance and overfitting behaviors.

As presented previously, the FTL can specifically tackle False Negatives and False Pos-

itives differently from class to class, according to the evidence gathered in the model evaluation phase. For example, if the class *Forest* displays a heavy number of False Negatives, then the α parameter for that single class can be incremented relatively to β . This approach can lead to beneficial outcomes during the training phase, resulting in a more punctual and rigorous strategy to prevent poor results in the validation set.

2.4.5 Optimizer and Learning Rate

The choice of both the optimizer and the learning rate differs from the solution proposed by [61]. In fact, while the authors proposed Stochastic Gradient Descent (SGD) with polynomial decay scheduler, during the experiments conducted throughout this project, it has been observed that the Nadam optimizer [22] converges faster. While the authors trained their models for 10k steps, time efficiency and faster convergence were vital elements in this project.

The objective function is thus optimized with the Nadam optimizer, which combines the Adam optimizer [33] and the Nesterov Momentum, and a learning rate scheduler that dynamically decreases as follows:

$$lr_{new} = \begin{cases} lr_{old} \cdot \text{factor} & \text{if no improvement for } p \text{ epochs} \\ lr_{old} & \text{otherwise} \end{cases} \quad (2.19)$$

where the initial learning rate is 0.001, the final learning rate is 0, and the value of lr_{new} gets updated if no improvement in *validation loss* is observed within 10 epochs.

While other momentum techniques consider past gradient values to speed up the optimization process, the Nesterov Momentum is a technique that accelerates the convergence to local or global minimum by involving a look-ahead approach, which is rather the opposite. Using an analogy, it allows a "projection into the future", as it does not compute the gradient at the current state, rather it computes the gradient at a future step based on the optimizer current trajectory.

The learning rate plays a fundamental role in deep learning models deployment, as too low values can lead to unsatisfying performances, whereas too high values can lead to exploding gradient phenomenon. In both situations, the training experience would be too unstable and the loss too large [6]. The choice of the initial learning rate value, as well as its decaying schemas of various nature, can profoundly impact the model in

two ways: first, it determines its ability to escape local minima in the initial phases of the optimization process by setting a relatively high learning rate; second, it avoids huge oscillation in the later stages of the process by lowering its value, so to ensure small update steps which (hopefully) help stabilizing the loss in a global minimum. Under this perspective, the learning rate can be interpreted as a trade-off between exploration and exploitation. Additional evidence [64] suggests that, in addition to model optimization, the learning rate is closely related with feature learning. More specifically, the initial, relatively high learning rates allow to learn more general patterns and avoid learning noisy information, purifying the learned patterns. The subsequent, decreasing learning rate, instead, enables the understanding of deeper, more complex patterns.

2.4.6 Class Weights

As mentioned in Section 1.2, the dataset suffers from class imbalance. One way to account for it, and hence avoiding the model to over-predict the most common classes, is to specify a vector of class weights. This will signal the model to give more importance to the least frequent classes. The obtained vector of weights, computed as a an inverse function of the number of pixels, is shown in Table 2.1:

Background	Building	Road	Water	Barren	Forest	Agriculture
0.37	1.34	2.81	2.32	2.84	0.92	0.73

Table 2.1: Class Weights for class imbalance

The formula deployed to obtain the weights is the following:

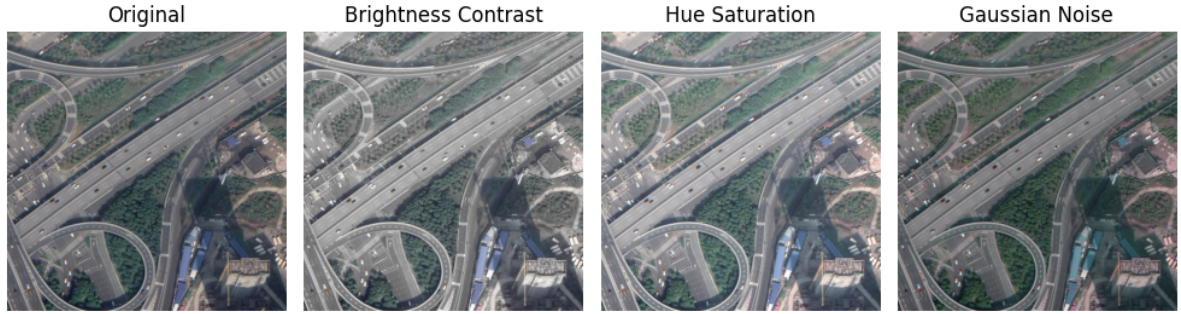
$$w_i = \frac{N}{7 \cdot P_i}$$

where w_i is the i -th class weight, N is the total number of pixels of all the classes, and P_i is the number of pixels for class i . For example, in the entire training set there are 2,644,508,672 pixels, of which 1,010,342,082 belong to the class Background. Hence, the class weight $w_{background}$ will be obtained as: $w_{background} = \frac{2,644,508,672}{(7 \cdot 1,010,342,082)} = 0.37$.

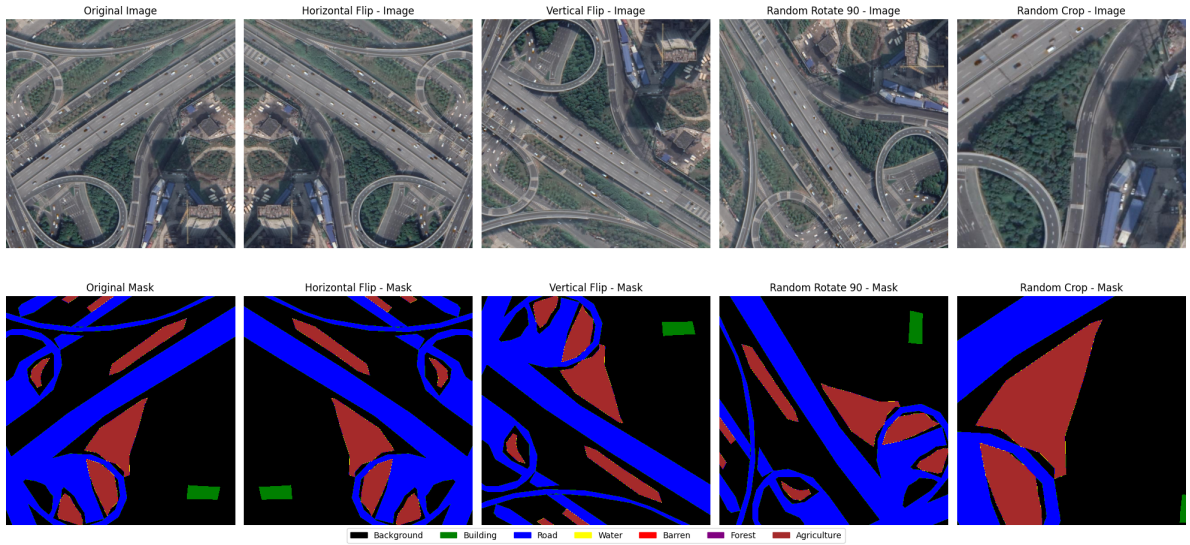
The weights assigned are inversely proportional to the pixel count of each class. The rationale behind this approach is that classes with fewer pixels (e.g. Barren) receive significantly higher weights. By assigning higher weights to less frequent classes, the model is less likely to become biased towards predicting only the majority class.

2.4.7 Augmentation Pipeline

Image augmentation allows to apply transformations to the training images (and their corresponding masks) in order to provide the model with a more heterogeneous input, making the model less prone to overfitting. This step is particularly useful when the dataset is not significantly large and results in a higher model generalization capability [53].



(a) Example of augmentation steps applied to the images only.



(b) Example of augmentation steps applied to both images and corresponding masks.

Figure 2.12: Augmentation steps applied to images only (top) and to both images and masks (bottom).

The augmentation steps, rigorously applied to the training instances only, are performed via the Albumentations library [10], which ensures the image and its corresponding mask receive the same exact transformation. The techniques applied in this

dataset are of two types: geometrical and color-based transformations. To the first group belong horizontal and vertical flips, random rotation of 90° , and random cropping. Such alterations are performed with respect to the (image, mask) pairs, as the transformation occurred to the image must be replicated in the mask as well, otherwise the pixel relationships would be lost.

To the second family of alterations belong random brightness contrast, hue saturation, and Gaussian noise. These have been applied to the images only, as the masks must maintain their (0,1) values. While the transformations of the first set were applied with a probability of $p = 0.5$, meaning that the selected modification is applied, on average, once every two images, the second group's techniques were applied with a probability of $p = 0.3$ in order to maintain a more cautious approach in data manipulation. While the first ones belong to the group of non destructive, spatial transformations, the last three entails non spatial ones. A visual example of the applied transformations is offered in Figure 2.12a and Figure 2.12b.

3 Popular Segmentation Architectures

This chapter illustrates the most widespread model architectures deployed in image semantic segmentation applications. It will guide the reader through the main peculiarities, strengths and weaknesses, and the novelties introduced by each of these modeling frameworks.

One notable group of models are based on encoder-decoder architecture in the general framework of convolutional neural networks. The *encoder* part, also called contracting path, roughly speaking the first half of the architecture, is responsible for decreasing the input dimensions while learning patterns via the feature maps. It is essentially what an image classification model does, except that in this case the final classification output layer is obviously not deployed. After the contracting path, there is the *decoder*, also called expanding path, which has the goal to restore the output of the encoder to the original input dimensions, and to produce a prediction for each pixel in the image. The decoder part is responsible for translating the learned feature maps into final predictions. Examples of models belonging to such family are discussed from Section 3.1 to 3.4.

Encoder engines are often formed by previously trained networks, hence forming a pre-trained *backbone* for the encoder-decoder model. Examples of commonly chosen pre-trained models are ResNet [29], Xception [16], and EfficientNet [57]. The rationale behind this approach is to leverage the already-proven ability of such networks to extract robust patterns for image classification tasks on very large datasets with heterogeneous class labels. The core idea of such practice is to exploit the generalization power these networks are equipped with. In Section 2.2 it was mentioned that one crucial characteristic of deep learning methods is that they can be applied to several different tasks without the need of hand-crafting special features, provided these models' ability to adapt and generalize feature learning. In this way, the segmentation model can build its learning abilities upon a strong foundation inherited from the pre-trained backbone.

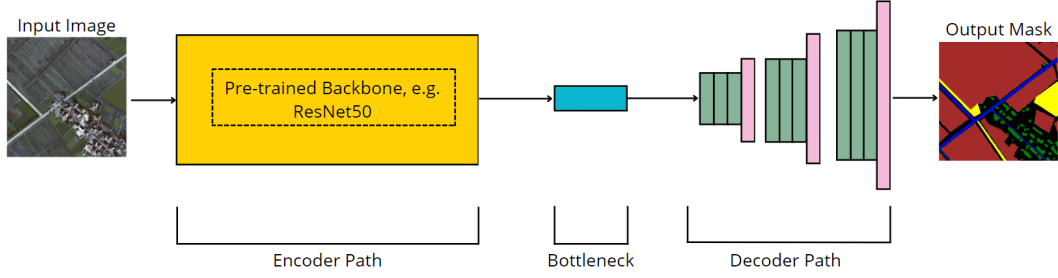


Figure 3.1: Example of encoder-decoder architecture with pre-trained backbone

This approach, represented in Figure 3.1, goes under the name of *transfer learning*, as its goal is to transfer the knowledge those models previously mastered to a new, diverse task. Such practice has two major advantages. First, it ensures precise and trustworthy feature extraction. Second, it accelerates the training, as the segmentation model, rather than learning from scratch, already starts with a set of previously learned patterns. The selected backbone can then undergo a process called *fine-tuning* to better adapt to the specific research case. This is done by *unfreezing* the final layers of the backbone. Technically, this means that a certain amount of layers at the end of the backbone structure are allowed to be trainable on the current task, meaning that their weights will be recursively updated during training, whereas all the previous layers are left unchanged and non-trainable. Such design guarantees that the segmentation model detects specific patterns without compromising the backbone’s foundational knowledge.

While Convolutional Neural Networks are the gold standard in many computer vision tasks including image segmentation, more recent studies highlighted the possibility of still deploying encoder-decoder architectures, yet not necessarily in the CNN framework, marking a paradigm shift. Such models are presented in Section 3.5.

3.1 Fully Convolutional Networks

The first study to propose the idea of deploying an encoder-decoder architecture in semantic segmentation applications came from Long et al. [39] with their Fully Convolutional Network (FCN). Their main intuition was to adapt already existing convolutional neural networks, traditionally used for image classification, to image segmentation, hence extending their usage to pixel-wise prediction tasks. To achieve this, they replaced the last fully connected layers of the CNN - which would collapse the spatial dimensions of

the feature maps - with convolutional layers having kernel of size 1×1 , which maintains the spatial structure of the feature maps. Another major contribution was the use of skip connections within the encoder part. Those combined outputs from earlier layers with deeper ones in order to maintain information about the spatial dimensions. Earlier layers are able to learn coarse, low-level features of the input image, whereas subsequent layers are capable of mapping more high-level, abstract structures. Moreover, the authors introduced the concept of upsampling the learned feature representations in the decoder part via bilinear interpolation, which is capable of restoring the original image input size while maintaining the learned patterns.

Since their release, FCNs lead the way to more refined, accurate, and increasingly precise models that have been inspired by the encoder-decoder layout.

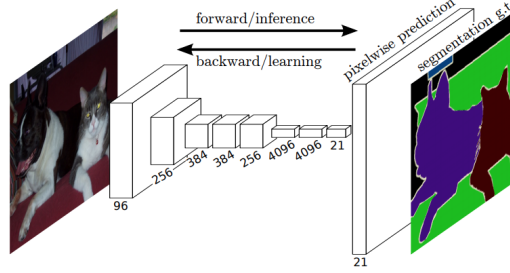


Figure 3.2: Fully Convolutional Network by Long et al. [39]

3.2 U-Net

Speaking of models influenced by the FCN innovation, U-Net [48] is certainly one of the most popular, and probably the one that imprinted a significant increase in the adoption of such model setup [54].

One significant change introduced by U-Net is that the expanding path is made up of *many* upsampling operations, each followed by a standard convolutional layer, differently from the FCN, which handles upsampling through one single step. It is vital to highlight the presence of convolutional layers immediately after the upsampling: these convolutions have the scope to *learn* how to assemble the output coming from the upsampling operation. More importantly, differently from the FCN, the upsampling in this architecture is performed via Transposed Convolutions. The difference between the two is conceptually important, as the upsampling is not equipped with learnable pa-

rameters whereas the transposed convolution does have learnable weights. Overall, the deeper structure of the decoder, paired with the (already) deep structure of the encoder, donates a symmetrical shape to the whole architecture.

Another significant difference with respect to FCN is the way skip connections are managed. In this case, the skip connections are placed so that they connect output layers of the encoder with output layers of the decoder. The main intuition is that in this way these connections directly pass feature maps from the encoder to the decoder, allowing the network to recover spatial details lost during downsampling. The restoring of the original input size can be achieved more accurately thanks to the better localization ability promoted by the skip connections. This profound difference can be observed when comparing Figure 3.2 with Figure 2.1.

Since it was firstly applied to a dataset with limited number of images, to compensate for this limitations the authors relied on intensive data augmentation techniques, whose beneficial outcomes have been anticipated in paragraph 2.4.7. More in detail, the augmentation pipeline utilized by the authors relies on elastic transformations, smooth deformations, and per-pixel displacement with bicubic interpolation.

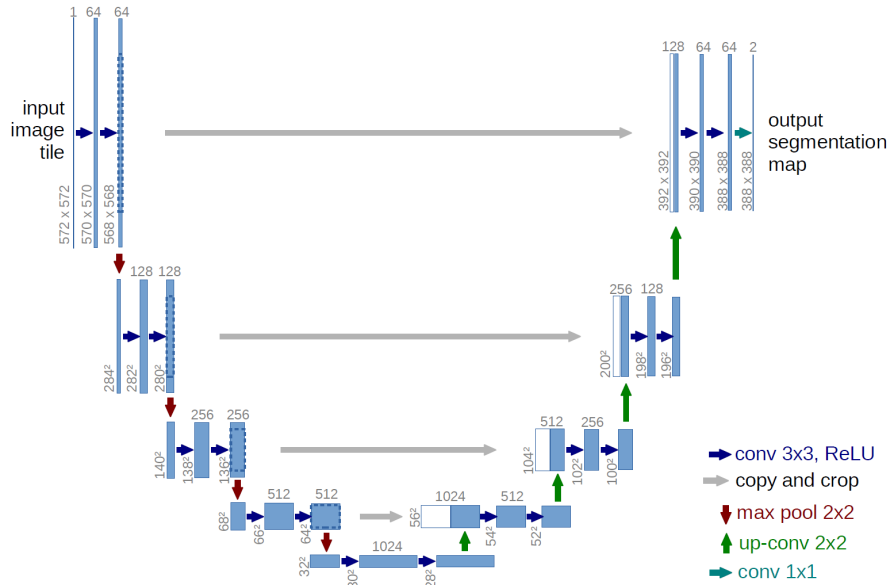


Figure 3.3: Original U-Net architecture by Ronneberger et al. [48]

The contracting path of U-Net consists of a sequence of 5 convolutional blocks, each made up by triples of 3x3 convolutions, ReLU activation function, and 2x2 max pooling.

The chosen number of filters doubles as the input goes through the network. The part in which the last convolutional block is executed goes after the name *bottleneck*. After this stage, the decoder part takes action. The bottleneck output is upsampled via 2x2 transposed convolutional layers and thus concatenated with the deepest skip connection, i.e. the output of the last convolutional block before the bottleneck phase. The concatenation operation is performed between two feature maps sharing the same spatial dimensions. After the concatenation, a 3x3 convolutional operation is applied along with the ReLU activation function. The number of filters gets halved as the decoder path grows, maintaining a coherent, mirrored relationship with its encoder counterparts. Such backwards operation is repeated until the original input shape is matched. The last operation is a 1x1 convolution with as many filters as the number of classes to segment the image into, followed by the sigmoid activation function, which will enable to obtain pixel-wise classification of the input image.

3.3 SegNet

SegNet is a fully convolutional network segmentation model with an encoder-decoder architecture proposed by Cipolla et al. [4] in 2017. The goal of the authors was to reach state-of-the-art segmentation accuracy while optimizing memory utilization and limiting the computational burden, with the goal to craft a model suitable for real-time tasks, such as road scene understanding. Although its initial applications were limited to indoor scenes or road scenes, it nonetheless represented an important milestone for the whole field of semantic segmentation in general.

The encoder phase of SegNet deploys a 13-layer deep VGG-16 architecture, without the last three fully connected layers. There are a total of five down-sampling blocks. The first two comprise a sequence of two convolution-ReLU pairs, then followed by a max pooling layer. The last three blocks are instead formed by a combination of three convolution-batch normalization-ReLU pairs, followed by max pooling. On the other part of the architecture, the decoder precisely mirrors the encoder structure, so that ideally each encoder layer has its own decoder layer counterpart, and each pooling layer has its corresponding upsampling operation. For example, the last encoder block consists of a sequence of three Convolution-Batch Normalization-ReLU triplets, hence in the corresponding decoder block, namely the first, there will be a corresponding triplet after the upsampling layer.

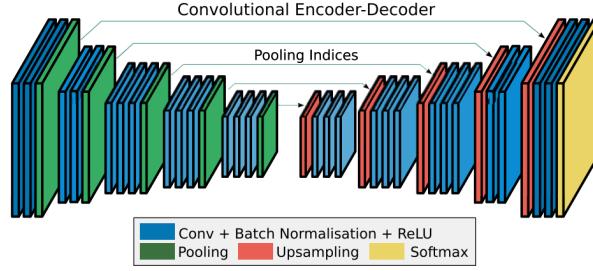


Figure 3.4: SegNet Architecture with the corresponding layers, by Cipolla et al. [4].

In SegNet, the authors observe that repeated combinations of convolutional and pooling layers allows to obtain rich, abstract feature maps, but on the other hand it causes to lose crucial details about the spatial relationships of the object within the input image. This is an acknowledged implication of this standard approach, and others have proposed their own solutions (see 3.4). While in [4] it is acknowledged that utilizing skip connection between encoder and decoder to transfer spatial resolution at different levels is a valuable approach, the authors claim that it would be too memory-intensive and computationally expensive for real-time applications such as road scene understanding.

The biggest novelty introduced in SegNet lies in the way the decoder part is implemented. The proposed innovation arose from the motivation to design a model that could improve upon its counterparts in terms of practicability and deployment efficiency. In summary, during the pooling operations in the *encoder* part, instead of retaining the entire feature maps after the pooling steps, and passing them to the corresponding decoder part for spatial resolution reconstruction (as seen in U-Net), this novel approach proposes to store the *indices* at which the pooling operates. During the decoder, the processed feature maps will be upsampled by exploiting the memorized pooling locations identified and stored while in the encoder part. By recording only the max pooling indices instead of the entire feature maps, SegNet successfully reduces memory usage without sacrificing spatial details.

To illustrate how the novel approach works, consider a 4×4 feature map that is subject to max pooling with a 2×2 kernel window and a stride of 2. As a result, the original feature map is reduced to a 2×2 feature map. Instead of looking at the maximum values of each 2×2 kernel window, the important information is the values' corresponding indices. Now, imagine that during the decoder phase a feature map of dimension 2×2 has to be upsampled to a 4×4 feature map. At this point, an empty 4×4 feature map is initialized with zeros. Then, the values of the 2×2 feature map will be placed in those *indices* that

have been identified in the pooling operation in the encoder. Figure 3.5 visually clarifies this process.

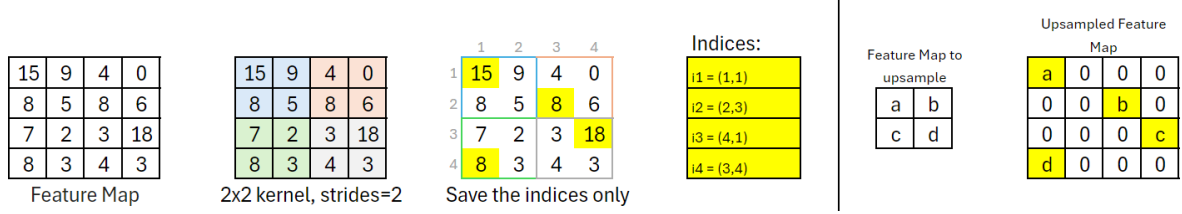


Figure 3.5: Pooling operation according to SegNet. The left-hand side refers to the pooling during the encoder. The right-hand side refers to the feature upsampling during the decoder.

When compared to similar solutions such as U-Net, one significant difference is that the former entirely transports the entire feature maps from encoder stage to corresponding decoder stage, whereas the latter transfers the indices.

3.4 DeepLab

Rather than a single model architecture, DeepLab is a family of models for semantic segmentation tasks developed by Google Research. There exist a total of four different versions, whose publications span from 2015 to 2018. This ensemble of models aimed at proposing solutions to the fact that there exist multi-scale object in an image subject to segmentation algorithms, and this unbalance can lead to poor learning.

The first three architectures combine a pre-trained encoder part, along with their peculiarities, with an upsampling phase handled by bilinear interpolation. Hence, they formally do not belong to the encoder-decoder category. However, the fourth and definitive model explicitly includes a decoder with transposed convolutions.

3.4.1 DeepLabV1

In the first DeepLab model, Chen et al. [12] identified two main limitations of Deep Convolutional Neural Networks applied to segmentation purposes. The first focuses on the reduced signal resolution caused by the repeated down-sampling operations, which is recurrently deployed to create deep feature learning. In other words, the fine-grained details are "lost" due to the down-sampling, which causes a waste of important information. The second problem relates to the fact that Deep Neural Networks are often

designed to learn patterns and motifs regardlessly from the position occupied by the objects in the image, a property called translation invariance, but this information is not negligible in semantic segmentation, where the relationship between objects locations can be crucial. A good segmentation model not only needs to identify *what* class certain pixels are part of, but also in which position of the image they are located, i.e. *where*. To address the first problem, the authors propose Dilated Convolutional Layers, also called Atrous Layers [65]. The goal of Atrous convolutions is to expand the field of view of the convolutional kernels. This is achieved by enlarging the kernel, augmenting it with additional holes. This is done by filling the kernel with zeros, similarly to what happens to the input feature map in the transposed convolution. Such layers were added after a pre-trained VGG16 backbone, which implied a sequence of standard convolutions and pooling layers, before the upsampling step.

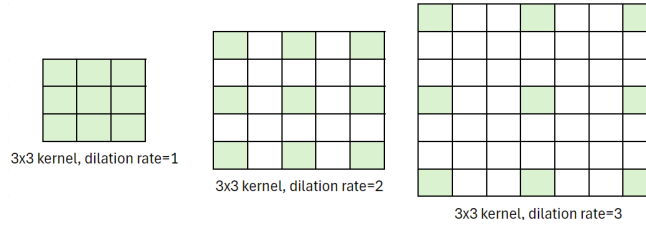


Figure 3.6: Dilated Convolution kernels with different rates (1,2,3)

On the other hand, to tackle the second issue, Conditional Random Fields (CRFs) were proposed as a post-processing step. Conditional random fields are applied after the predicted masks are obtained, and the main principle is to compare the pixels located at the boundaries between two different classes, where the network might be highly uncertain about, and investigate their similarity. If the selected pixels are deemed similar by the algorithm, then it is likely that they belong to the same class, even though they might have been classified oppositely from the Deep Network. Conversely, if two pixels show significant different pixels characteristics, they will be assigned to two different classes. Conditional random fields can be seen as a refinement tool, applied after a model has been trained and tested, rather than a segmentation technique itself [56].

3.4.2 DeepLabV2

The second proposed version [14] improves upon its previous counterpart by including a *block* of dilated convolutions with different dilation rates. More precisely, a single

block was formed by many Atrous convolutions, usually four, with increasing dilation rates. Such block went under the name of Atrous Spatial Pyramid Pooling (ASPP) and its role within the entire architecture was placed at the end of the encoder part. The encoder is also modified, as it was selected a pre-trained ResNet-101. The core idea behind incorporating Atrous convolutions with multiple dilation rates is that higher the dilation rate, more general will be the detected pattern. This allows to parallelly capture large-, mid-, and small-scale objects. Roughly speaking, larger dilation rates capture broader context, whereas smaller dilation rates detect finer details. A concatenation phase is then applied to merge the learned, heterogeneous feature maps, before feeding them to the bilinear upsampling phase.

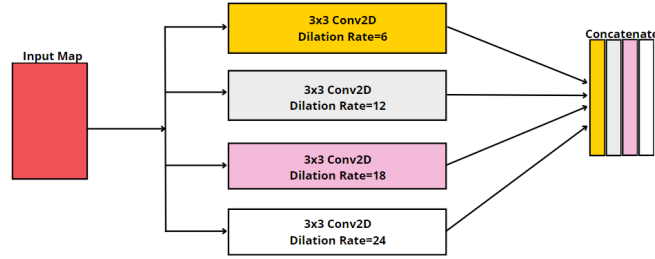


Figure 3.7: Atrous Spatial Pyramid Pooling (ASPP) block, placed at the end of the encoder. The concatenated result is then upsampled to obtain the desired output shape.

3.4.3 DeepLabV3

The third version [13] continued onto the direction marked by the Atrous Spatial Pyramid Pooling, but for the first time CRFs were removed. The goal was to build an end-to-end segmentation model capable of reaching state-of-the-art performances without relying on post-processing steps. In this version multiple pre-trained backbones have been experimented, included the already mentioned ResNet101 and the Xception. However, the backbone incorporated the ASPP modules, instead of appending it to the end of the encoder as an intermediate step before upsampling. An additional integration relates to the Global average Pooling layer placed after the concatenation, which has the goal to capture broad contextual information of the input image. Another important innovation is that Batch Normalization has been included after each dilated convolution within each ASPP block. As in the previous two versions, after the encoder part the feature maps undergone an upsampling stage managed by bilinear interpolation.

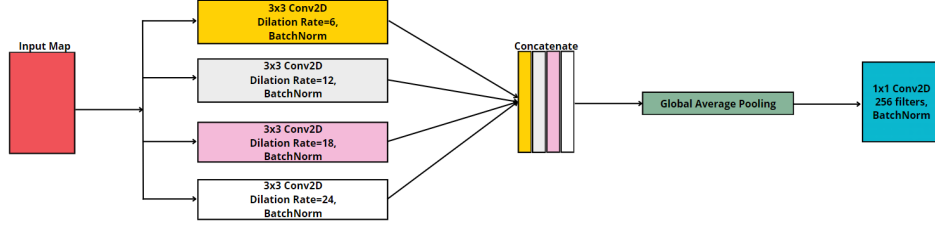


Figure 3.8: Refined ASPP block. After the last step, colored in blue, upsampling is performed to obtain the desired output shape.

3.4.4 DeepLabV3+

The last, most updated version of DeepLab introduces a significant innovation in how the feature maps are brought back to the original dimensions, i.e. the decoder. In fact, differently from the other versions, the currently discussed architecture includes a decoder part, making this model an encoder-decoder architecture. The encoder path is equivalent to DeepLabV3, meaning that it combines a modified Xception backbone that incorporates Atrous convolutions. The decoder handles a gradual upsampling of the feature maps. It combines high-level features, captured by deep feature maps obtained in deeper parts of the encoder, with low-level features (obtained from earlier stages of the latter), that are upsampled so to ensure dimensions matching during concatenation. By performing a gradual upsampling instead of in one single shot allows to obtain more accurate results.

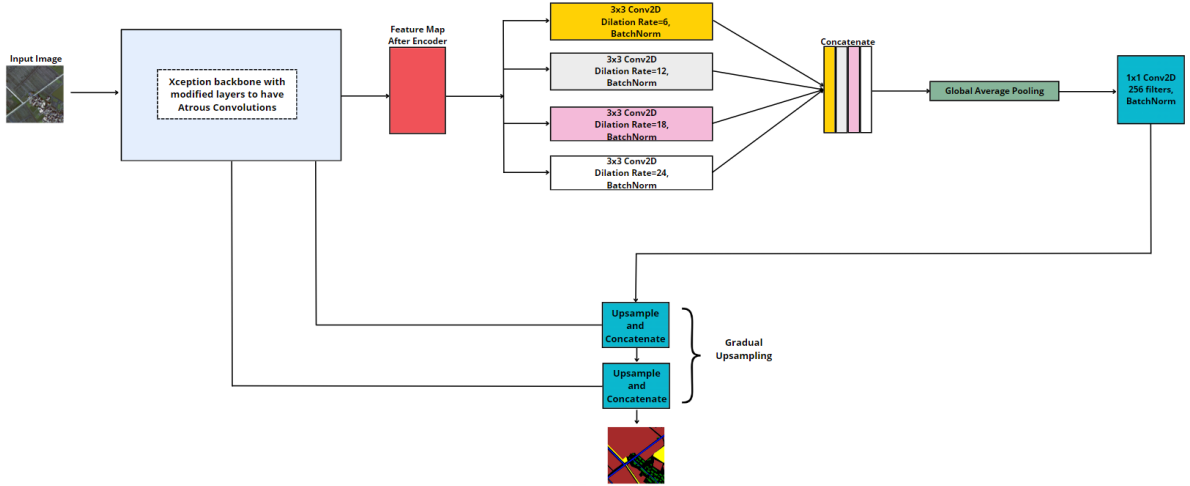


Figure 3.9: DeepLabV3+ complete architecture.

3.5 Attention Mechanisms-based Architectures

Attention-based techniques are one of the most promising area of research that emerged in the recent years in the field of semantic segmentation. While initially originated from the field of Natural Language Processing (NLP), attention-based methods have been subsequently extended to Computer Vision tasks.

As discussed in the dedicated sections, traditional Convolutional Neural Networks repeatedly apply a fixed $k \times k$ kernel size during the image processing steps designed to perform feature generalization learning. If, on the one hand, this inherent characteristic allows to grasp fine-grained, local details, on the other hand it can also represent a constraint, because the receptive field is limited to the kernel window and it does not empower the network to detect long range dependencies, possibly leading to a scarce understanding of the overall input image and its global context. This limitation is particularly pronounced in segmentation, where the spatial relationships between distant parts of an image can be crucial. For example, it may be important to know that in areas closer to a water body it is less likely to observe a road, whereas it is more likely to observe vegetation and thus forest or agricultural areas.

The fundamental idea at the basis of attention methods is to create a tool that is capable of allocating the proper focus (i.e. attention) to the most relevant areas of the image. This methods aim at replicating one natural behavior of the human eye. In fact, in the human visual cognition system, the eye is used to focus on the area of interest of any given object, while disregarding the unnecessary or redundant information [63]. Equivalently, the main intuition is that, in a given image, some areas, and hence the corresponding pixels, are more informative than other areas with respect to the final prediction. All the other information complementary to these most relevant regions may be redundant or misleading for the network, as it could learn noisy, unrepresentative relationships and thus be distorted and misguided during the learning process. Hence, attention methods try to offer a solution to mitigate such limitation, by emphasizing key regions and discard the less relevant areas.

This section will continue by introducing the functioning and the necessary background to understand an attention mechanism in the context of computer vision and will provide one example of convolutional neural network enhanced by the usage of attention blocks.

3.5.1 Understanding Attention's Functioning

The attention mechanism comprises three elements: Query (Q), Key (K), and Value (V). A preliminary phase before obtaining these three quantities implies to apply different convolutional layers, in order to obtain intermediate feature maps. These feature maps serve as the foundation for computing Q, K, and V and allows the model to work with rich feature representations while reducing computational requirements, as a pixel-wise application of the attention mechanism would be too computationally demanding. As a compromise, the attention is computed with respect to the feature map's positions, also referred to as patches interchangeably.

Firstly, the Query vector of each feature map's position represents what each single patch in an image is *looking for* in the rest of the feature map. The Query vector can be interpreted as a signal that defines what information each pixel wants to retrieve from other parts of the image, in order to assess which other pixels share the same or similar information. The obtained feature map is passed through a linear transformation, often achieved by a 1×1 convolution with d filters [43], to produce a query vector. This transformation allows each position in the feature map to have its own query vector.

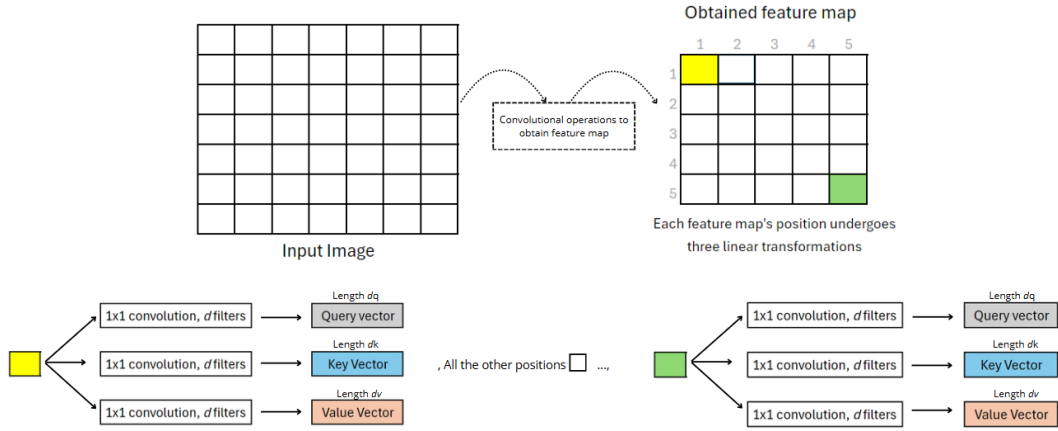


Figure 3.10: Visual Description of (Q,K,V) Vector Embeddings for each patch in the feature map. For Brevity, the first and the last only are reported.

The interpretation of such operation is that it enables each patch to "ask" what other positions of the image should it focus on, to improve its understanding from the rest of the image. The process of obtaining the Query, Key and Value vectors is called vector embedding. The dimensionality of the embedding vectors, d , is a hyperparameter chosen based on the given problem complexity and other tasks-specific characteristics,

and its choice can be compared to the choice of the number of filters in a Convolutional Neural Network convolutional block. By stacking all individual position's Query vector it is possible to obtain the Query matrix, and the same holds for the Key and Value matrices. Figure 3.10 illustrates how the Query vectors (as well as the Key and Value ones) are obtained, while Figure 3.11 shows how the respective matrices are derived.

The Key vector for each feature map's patch, also obtained with a linear projection, aims at encoding the content, the characteristics, of that specific location in the feature map. Each position's Key vector serves as a way to describe itself to other patches that may find it relevant based on their respective queries. Keys essentially provide a descriptor for each position, allowing other parts of the feature map (prompted by their queries) to decide if this region is relevant. Keys act as a reference for other positions query to compare against. In fact, when a position's query interacts with the keys of another region, it effectively compares itself against the characteristics of the entire feature map, allowing it to identify relevant regions for improved segmentation decisions.

Lastly, the Value vector, equivalently obtained through a linear transformation, carries the actual information or features of its associated pixels that other pixels belonging to other regions may need. In the attention mechanism, the values represent the information that is selectively aggregated based on the attention scores derived from the query-key interactions. Similarly to the K and Q counterparts, the Value matrix V can be obtained by stacking all the Value vectors.

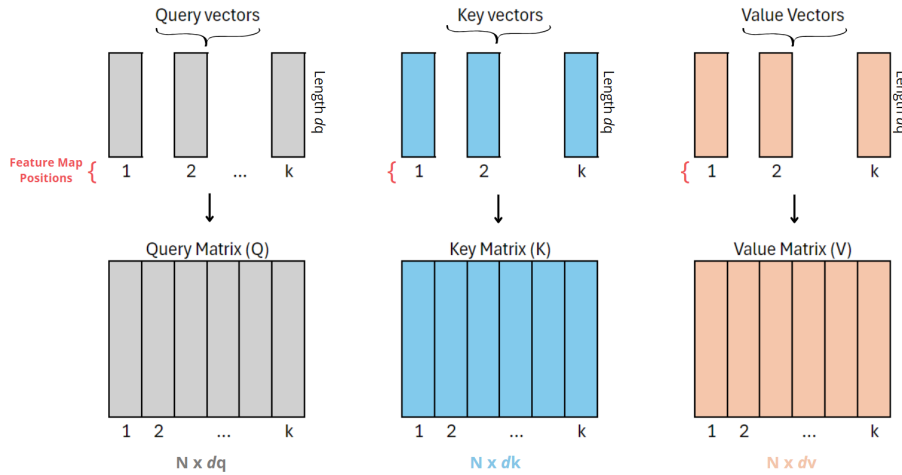


Figure 3.11: Creation of the matrices Q,V,K from the single vector embeddings.

Once the triplet (Q,K,V) is obtained, the attention scores are computed following Equation 3.1:

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V \quad (3.1)$$

The core of the attention mechanism lies in the interaction between Q and K. Each Query is compared to all Keys by computing the dot product between matrix Q and matrix K. This produces a raw attention score for each (position, position) pair, each score representing how relevant one pixel is to another based on the query's focus. This intermediate result can be compared to a heatmap matrix, in which high values resulted from the dot product signal high associativity between the two interested patches involved in the dot product. For example, if in position (1,4) of the raw attention scores matrix a high value is obtained, this means that the patch (1,1) and its counterpart (1,4) are deemed to have a high degree of similarity. In general, for a given row of the intermediate attention scores, each entry captures the similarity scale between the patch represented by the row index and the patch represented by the column index.

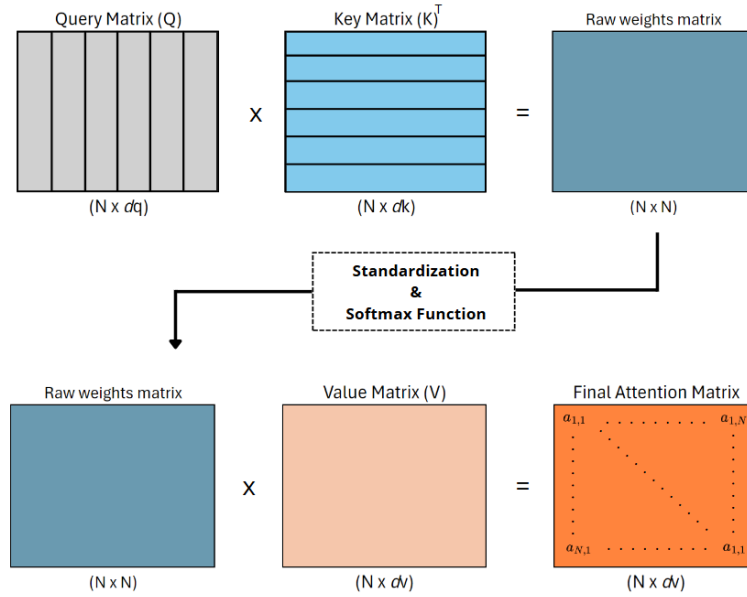


Figure 3.12: Visual Representation of the mathematical steps involving the matrices Q,K,V.

The raw attention scores are then normalized by the scaling factor $\sqrt{d_k}$ before being passed through a Softmax function. This last step enables to normalize the weights

into a probability distribution, as the matrix becomes row-stochastic. In other words, the obtained normalized matrix contains indication on how much weight, attention one patch should give to another patch. At last, the normalized and softmaxed raw scores are multiplied by the Value matrix. This ultimate step performs an aggregation between the weights and the value of each corresponding patch. So, a patch that was deemed similar with another will multiply that attention weight by the other patch's value, finally measuring the information to attend by. The whole process is depicted in Figure 3.12.

The process carefully presented and depicted in the previous lines of this chapter aims at exemplifying the core aspects of one type of attention mechanism, the so called Self-Attention. In fact, this is only one among the multiple ways the concept of *focus* can be defined into an Attention mechanism module [42]. For brevity, the Self-Attention modality will remain the only method to be discussed in depth, whereas the other two methods, Spatial and Channel attention, will be briefly introduced, mainly to clarify their different purposes. As the term suggests, Self-Attention aims at capturing the interdependence between different areas of the image, hence the focus in this case is directed to the relative similarity between those areas.

Differently, Channel Attention concentrates on the relevant channels, i.e. the feature map's depths. As it was mentioned in Section 2.3.2, each channel, also referred to as filter, in any feature map ideally captures information about one particular feature. In summary, Channel attention's goal is to prioritize the relevant channels and disregard the less relevant one. As a result, the most impactful features will be assigned higher attention scores. For example, the Channel attention tries to answer a question such as "Which features should receive more importance, the edges or the color brightness?"

The third and last focus mechanism in attention methods is the Spatial Attention. This attention process aims at understanding the relevant portions of an image, independently from the others and from the channels. It differs from the self-attention as the latter considers pairwise (patch, patch) relative relationships, whereas the former retrieves which locations the model should prioritize. An example question the Spatial attention cares about is: "Which portions of the image are relevant to this task, top-left corner, or center area?".

3.5.2 Attention U-Net

A significant contribution concerning Attention-based mechanisms materialized in 2018 when Oktay et al. [43] proposed a novel architecture for semantic segmentation of pancreas images that combined the standard, usual U-Net framework with the attention mechanisms. Despite it has been firstly applied to medical imagery, it can be extended to a variety of segmentation tasks, as its target is to learn to pay attention to object of varying sizes.

The motivation from which the design of the current architecture originated is that during the skip connections of the U-Net, it is possible that some coarse, redundant low-level features are transmitted to the decoder path as well. Such behavior can lead to learn noisy, redundant patterns and thus undermine the desired learning process.

The proposed solution uses additive soft attention [42] in the form of Attention Gates. In general, it involves to utilize a standard encoder engine, which can involve a pre-trained backbone, and to modify the way skip connections are managed with respect to a standard U-Net architecture. Recalling the usual skip connections process described in Section 3.2 when referring to U-Nets [48], that structure implied that the skip connections transferred progressively higher-level feature maps to the decoder part, with the goal to provide a more refined reconstruction of the input dimensions with localized segmentation masks.

Now, each Attention Gate, placed at each skip connection intersection with the decoder, controls feature transfer differently. Technically speaking, each Attention Gate takes as inputs an intermediate feature map x , located in the encoder, and a gating signal g , that comes from the decoder part. The idea is to compute attention weights to the input feature map x via the gating g , so that only the relevant features will be assigned high attention coefficients, avoiding to trail the ineffective feature maps from the earlier stages. This process is applied to multiple (feature map, gating signals) pairs along the encoder-decoder setup. As a result, each Attention Gate has its own attention scores, which capture which positions of the feature maps it is worth focusing on.

Like any neural network layer, each Attention Gate has trainable parameter, i.e. the weights involved in the linear transformations used to generate the attention scores. Hence, the entire attention mechanism, including each Attention Gate, is trainable via backpropagation. The overall architecture is depicted in Figure 3.13.

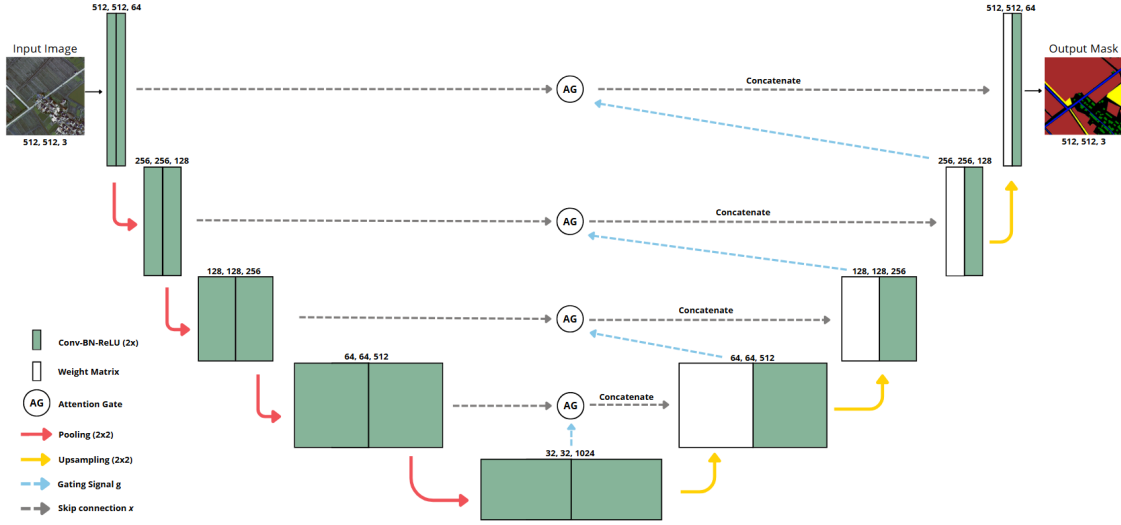


Figure 3.13: Attention Block in Attention U-Net

By controlling which features are passed from the encoder to the decoder, the gate effectively combines spatial information from low-level features with the semantic richness of high-level features from deeper layers.

More precisely, by zooming in into the Attention Gate's dynamics it is possible to accurately explain how it works.

The components coming from the intermediate feature maps, called x , are taken from one step earlier with respect to the corresponding gating signals g . Given these two inputs, a sequence of successive operation is performed. Since the gating signals come from one step deeper, they have lower spatial dimensions than the corresponding skip connection. So, if the skip connection has shape $(128, 128)$, its corresponding gating signal will be of shape $(64, 64)$. For this reason, the first step consists of aligning the two inputs on the same shape.

To achieve a state in which both x and g are of the same height and width, it is necessary to apply a (1×1) convolutional layer with $\text{stride} = (2, 2)$ to the x input, as the stride will halve the input feature map, hence making it matching with the g counterpart. The number of filters in this convolutional operation is a hyperparameter, and it is usually equal to the number of filters of the x input. The gating signal g will instead undergo a (1×1) convolutional operation with $\text{stride} = 1$, preserving its height and width so they match with the convolved x and the same number of filters is chosen.

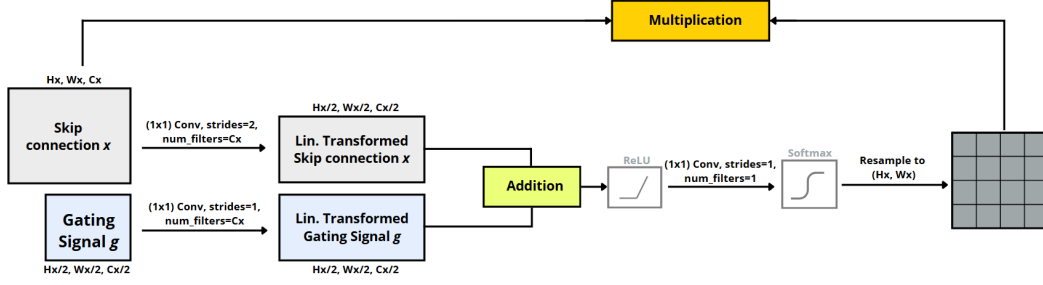


Figure 3.14: Attention Gate's Dynamics

Once the dimensions match, the two elements can be added. This step can be described as a feature alignment step. The intuition behind addition is that the position in the feature map that were important in one input, and are important in the other input as well, will be added, magnifying that that specific position is deserving a high weight. The obtained added feature maps are submitted to a ReLU activation function. Such operation ensures that position with weights lower than 0 are muted, whereas position with positive values are assigned a linearly positive value.

Subsequently, the tensor is passed through a (1x1) convolution with only one filter. Hence it will have shape (height, width, 1). This obtained matrix contains nothing but the attention weights for each position in the merged feature maps of that specific attention gate.

Since the ReLU activation function was deployed to discard meaningless activations, some values can be extremely high, and thus they need to be constrained to a fixed interval of values. Such thing is provided by the Softmax function which also ensures that the obtained results can be considered as probability distribution.

The final step is to upsample the obtained attention weight matrix to match the original input x size. This is necessary as the input x and the upsampled weights map are multiplied element-wise, which ensures that the original input feature map coming from the skip connection is aggregated by the corresponding attention score. In other words, the obtained attention weight matrix plays as a scalar parameter which magnifies the role of relevant features and suppresses the role of irrelevant ones before feeding them to the decoder. This flow is depicted in Figure 3.14.

Compared with the normal version of the U-Net, this enhanced version recursively select which features are worth learning and transferring during the skip connections, instead of transferring all of them passively.

4 Implemented Solutions

In this chapter, the implemented architectures will be analyzed and discussed. Some of the presented architectures in Section 3 have been tested on the current research, whereas the remaining will be left to future experiments. The proposed models range from a vanilla U-Net to a more sophisticated model like the Attention U-Net, passing through implementations of DeepLab and U-Nets engines with pre-trained backbones.

All the models have been tested at equal input size. At first, an input size of (128,128) was chosen to limit the computational burden during the first exploratory experiments and limit the amount of time necessary to assess the potential of each model. It has been successively increased to (256,256) and finally to (512,512) marking the definitive input size across all the definitive proposed models.

All the proposed solutions have been trained with an Early Stopping policy, meaning that, in case after a given number of epochs (that is, the patience parameter) during which the validation loss didn't improve, then the training would automatically stop. Such choice allowed to assess the diverse models while minimizing the time required for training. As a consequence of this choice, the proposed models have been trained on a heterogeneous number of epochs: those converging pretty quickly to their best potential are associated with a lower number of epochs, whereas other models required a longer training time.

In case pre-trained backbones have been utilized as the encoder part of an architecture, the corresponding proper input preprocessing has been carried out. In fact, different backbones may require different specific input formats, such as distinct scaling and normalization. For example, if a ResNet model is selected as backbone, then the input shall be converted from RGB to BGR and zero-centered with respect to the ImageNet dataset, without scaling. This ensures that the input data of the proposed implementation adheres with the input data the original backbone demands. When no pre-trained backbone is deployed, the input data is standard scaled to the $[0,1]$ interval.

Each model, representing the best realization of its corresponding model class, will be presented in its details, choice of hyperparameters and will be provided an assessment of

the produced estimates on the validation set, especially focused on the learning curves behaviors. A final comparison will allow a transversal evaluation across all the models.

4.1 Baseline U-Net

The first model being tested is a Vanilla U-Net with five encoder and five decoder blocks, comprising a total of 86 layers including convolution, pooling, batch normalization, and activation layers. This model consists of over 15 million trainable parameters. This model serves as a simple benchmark as it will represent a simple, shallow model with respect to the subsequent ones. The selected architecture replicates the main ideas proposed by the original paper [48], although some modifications have been introduced.

First, Max Pooling was replaced by Average Pooling. This ensures that smoother feature maps are obtained through the down-sizing operations. One common detail is the selected kernel size for the pooling, namely (2x2).

In the original paper, the padding was set to "valid", meaning that there were no additions on the boundaries of the feature maps. In the solution hereby proposed, padding is included to ensure that the input and output feature maps during a convolutional layer have the same spatial dimensions, before being halved by the successive pooling layer step.

One additional difference relates to the chosen kernel weights initializer. While in the original U-Net a Gaussian distribution with $\sqrt{2/N}$ was chosen, where N is the number of input nodes, in this version the He initializer [28] has been chosen, as it is considered more beneficial for deeper networks and models using ReLU activations.

Batch normalization layers are included after each convolution and before activation in both the encoder and decoder. This stabilizes the training and is a strategy not pursued by the original paper.

Finally, dropout layer have been replaced by spatial dropout layers, with rate of 0.2, to focus the dropout on the feature maps rather than on the neurons.

In the original version the skip connections were transferred and cropped before being concatenated to the corresponding decoder block. The cropping was necessary as it is given by the fact that the spatial dimensions of the feature maps were shrunk as a consequence of the chosen padding strategy. However, in this project, the goal is to retain the original input size, so skip connections are added without cropping, ensuring that they match the spatial dimensions of their corresponding decoder block directly.

As a result, a skip connection and its corresponding decoder part already are of the matching sizes.

4.1.1 Model Performance

An immediate tool for evaluating one given model's performance is through a thorough analysis of the recorded loss and metrics values, not only in terms of their raw, final values but also considering the overall trajectory of the respective learning curves.

By simply examining the training and validation loss curves over the epochs (Figure 4.1), it is possible to observe a desirable decreasing trend affecting both the curves. Such pattern is not rare and it indicates that the model is actively learning from the input data. However, the training loss reaches lower values of the loss function, whereas the the gap with the validation loss is subtly increasing as the epochs go by. Such behavior may be a symptom of overfitting. However, since the validation loss continues its downward trend even at the end of the training steps, there is potential for further learning and performance improvements with adjustments.

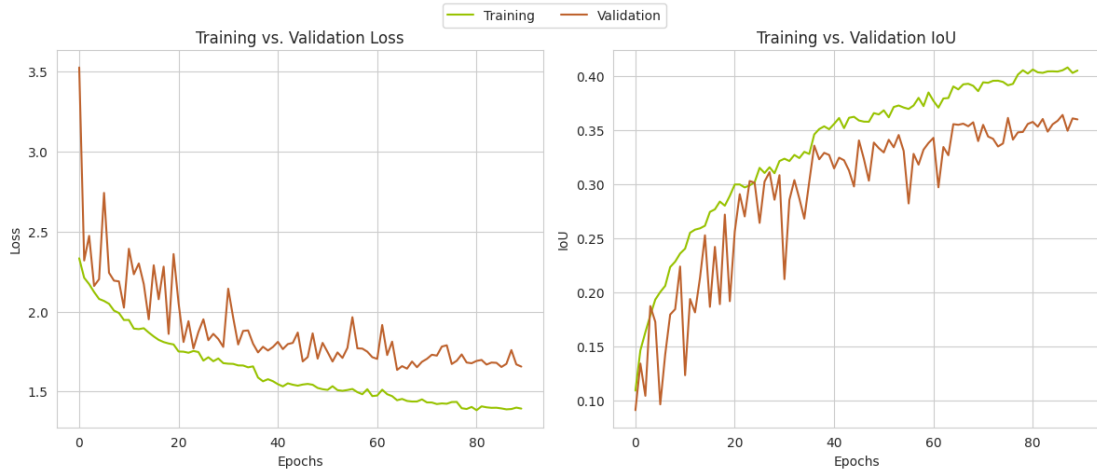


Figure 4.1: Baseline U-Net Results

Around the 60th epoch, a period of mild volatility is observed, as the validation loss increased but subsequently resumed to follow its downward path. Such fluctuations are common and may be due to batch variance. In general, it is a good sign that the model uncertainty phase is restricted to few intermediate epochs.

A similar discussion can be written with respect to the Intersection over Union (IoU)

metric. As expected, the training IoU is higher than the validation counterpart. A pleasing characteristic is that both the curves show an ascending trend during the entirety of the training periods, confirming that the model could have achieved higher scores by simply extending the learning phase. The two curves register a gap of 5 decimal points in the IoU metric. This model achieves a maximum Mean IoU score of 0.36 in its last epoch. Such result can represent a good starting point for a cheap, shallow model like Vanilla U-Net.

As regards the model IoU performance across each single class, a summary snapshot is provided in Table 4.1. Classes such as Building, Road, Water, Barren, and Agriculture (to a lower extent) show solid performances, as the training and validation results are almost perfectly equivalent. However, the Forest class is associated with weak results as the the validation result is significantly lower than its training companion, exceeding a value of above 0.20. One positive result, instead, is provided in the class Background, for which the validation IoU is above the training IoU. This pattern could indicate that the model finds it easier to identify background regions in unseen data than in the training set.

Overall, this model offers sufficient results. Nonetheless, the results suggest that the model exhibits good generalization ability in certain classes, yet its global performances are, as expected for a baseline model, lower than the state-of-the-art results.

	Background	Building	Road	Water	Barren	Forest	Agriculture	Overall
Validation	0.32	0.38	0.41	0.51	0.21	0.28	0.40	0.36
Training	0.27	0.39	0.39	0.53	0.24	0.52	0.48	0.41

Table 4.1: Vanilla U-Net IoU scores, single-class and overall, on the validation set.

4.2 Pre-trained Backbone U-Net

While the vanilla U-Net architecture demonstrates decent performance regarding the behavior of the learning curves and their final values, the objective now is to enhance the model’s structure to reach more competitive results. To accomplish this, a pre-trained backbone has been introduced as the encoder part of the network. The benefits of transfer learning have been discussed in the initial part of Chapter 3.

A total of 34 experiments have been conducted with pre-trained backbones U-Net. Such experiments had the goal to tweak different model’s characteristics such as the

learning rate inner parameters, potential regularization adjustments, different loss functions mentioned in 2.4.4, augmentation transformations in the dedicated pipeline, diverse backbones, optimal number of trainable layers, and input size. This group of experiments represent the most extensive efforts conducted in trying to replicate state-of-the-art results concerning the Convolutional Neural Network segment.

4.2.1 ResNet50 Backbone

The first chosen backbone is a ResNet50 model [29]. It was chosen because it has an established reputation of being an effective feature extractor and it represents a good trade-off between model complexity, the number of trainable and non-trainable parameters, and the expected output quality.

The backbone alone comprises a total of 173 layers, marking a significant increase with respect to the Vanilla U-Net depth. Based on the empirical evidence gathered, the optimal configuration involves setting the first 143 layers as non-trainable, leaving the remaining 30 layers as trainable. This allows to utilize both generalized knowledge learned from external pre-training, and task-specific learning refined in those last 30 layers. Overall, this model has 48 million trainable parameters.



Figure 4.2: ResNet50-UNet Results

The analysis of the loss curves shows that the training loss has a clear, steady decreasing trajectory, whereas the loss curve associated with the validation data follows a decreasing trend in the first 15 epochs, reaching a minimum value of 1.30, and successively it slightly increases before fluctuating around a mean value of 1.45. Moreover, the

training loss appears to be more stable, less volatile when compared with its validation counterpart. This suggests that the model to some extent struggles in generalizing the learned patterns to the unseen images and masks. In general, these two behaviors taken together, imply that the model is suffering from overfitting. In other words, it learned the training data increasingly well, probably too much, without being able to replicate the same behavior on the validation instances.

The right-hand panel of Figure 4.2 displays the Mean IoU for training and validation data. Complementarily to what has been written a few lines above, the training Mean IoU increases at a steady rate, showing almost no volatility. This suggests that the model is constantly learning from the input training images, approaching and then surpassing a Mean IoU value of 0.60. Conversely, after a period of increasing trajectory, the validation Mean IoU stabilizes just below 0.50, not without a few peaks and troughs. Despite there does exist an overall upward trend, the validation metric does not mimics the training values. This inconsistency reflects the model’s difficulties in achieving stable performance on the validation set, supporting the diagnosis of overfitting. Nonetheless, the *levels* this model reached in terms of minimum loss and maximum IoU are notably on a better scale, when compared to the Vanilla U-Net model: moving from a validation Mean IoU of 0.36 to 0.49 marks a substantial improvement.

	Background	Building	Road	Water	Barren	Forest	Agriculture	Overall
Validation	0.53	0.53	0.50	0.68	0.29	0.29	0.62	0.49
Training	0.56	0.55	0.60	0.73	0.47	0.63	0.71	0.61

Table 4.2: ResNet50 U-Net IoU scores, single-class and overall, on the validation set.

In Table 4.2 the single class IoU are reported. The classes Building, Water, and Background are confirmed to display a good similarity between the validation and training results. Road and Agriculture have a slight larger difference between the two values, yet they can still be deemed as satisfactory results. Similarly to what observed in the Vanilla U-Net model, the class where the model clearly fails in generalizing is Forest. One negative note entails the Barren class: also in this case the model learns too well the training data while it does not work as well in the validation instances. Such result was expected before the experimenting phase, as a similar poor result is obtained by the dataset’s authors as well [61]. Despite the deteriorated performance concerning pixels belonging to Barren, and the persisting difficulty concerning Forest, the model significantly improves its performance with respect to the other classes, where it registers

remarkable improvements often surpassing a magnitude of 0.20.

4.2.2 EfficientNetB5 Backbone

An alternative backbone that has been selected and deployed during the 34 experiments concerning the U-Net with pre-trained encoder is the EfficientNetB5. The choice is motivated by the fact that this backbone is associated with higher results on the ImageNet competition [57]. In fact, if ResNet50 achieved a top-1 accuracy of 74.9%, the EfficientNetB5 model is associated with a 83.6% for the same evaluation metric¹. Therefore it could represent a tool for potential improvement in the overall model performance of the current thesis project, and it could provide a better generalization capability to the data, hopefully minimizing the overfitting encountered in the ResNet50-based U-Net.

The EfficientNetB5 comprises a total of 570 layers, including resizing, multiplication, addition, other than the usual convolution, pooling, activation, batch normalization, and dropout layers. If the number of layers is significantly higher than in ResNet50, the number of trainable parameters of the two models is within the same order of magnitude. Similarly to ResNet50, for the EfficientNetB5 it is possible to specify the amount of layers that are allowed to update their weights, and hence being trainable, and the amount of layers that have to be considered frozen. After several experiments, the optimal number of trainable layers was settled at 70, hence freezing the first 500. In total, this model has 46 million trainable parameters.

Given the expectations of an enhanced performance, the U-Net with pre-trained EfficientNetB5 encoder offers disappointing results. The training and validation loss curves exhibit the same properties as in the previous models' experiments, however this time the levels reached are definitively worse. The validation loss scores similar results when compared with the Vanilla U-Net, with the difference that in this case there is no downward trend. Similarly, the validation Mean IoU results achieved by this model represent the worst seen so far. The EfficientNet-based U-Net is surprisingly the worst model under every perspective. Different setups and experiments have been conducted, concerning the backbone's properties, yet the best results of this model, reported above, are simply extremely terrible.

¹As reported by <https://keras.io/api/applications/>

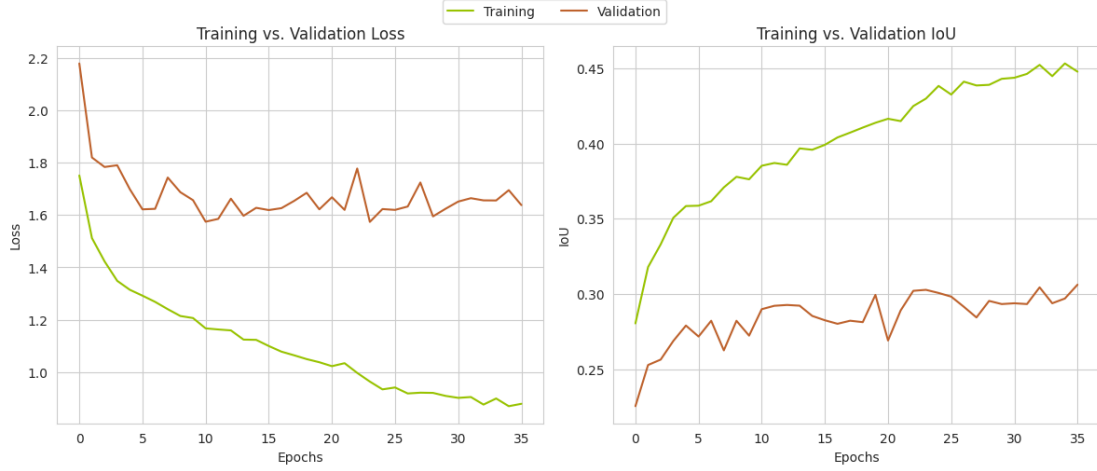


Figure 4.3: EfficientNet U-Net Results

	Background	Building	Road	Water	Barren	Forest	Agriculture	Overall
Validation	0.48	0.53	0.22	0.06	0.26	0.21	0.41	0.31
Training	0.57	0.49	0.25	0.18	0.58	0.56	0.63	0.46

Table 4.3: EfficientNet U-Net IoU scores, single-class and overall, on the validation set.

A closer inspection into the model's performance in each single class (Table 4.3) allows to understand precisely where the model failed. One number that immediately distinguishes from the other is the result on the Water class. It is surprisingly the lowest recorded value, whereas in both the Vanilla U-Net and ResNet50 U-Net the Water class was the category having the highest training and validation IoU. Moreover, a similar comment can be conducted with respect to the Road class. Even in this case the model performance is terrible. Such results indicate that the model fails at even grasping the patterns relate to these two classes. One possible interpretation is that the inherent characteristic of the EfficientNetB5 model are not suitable for the understanding of narrow, elongated figures (Road) as well as broad, spacious objects (Water). Such interpretation is backed by the additional evidence coming from the Agriculture results. In fact, the agricultural areas are often wide, large objects as in the Water case, and also in this class the performance is lower than expected, although to a lower extent when compared to Road and Water.

4.3 DeepLabV3+

Having established that pre-trained backbones yield superior results, specifically when the chosen backbone belongs to the ResNet family, the DeepLabV3+ implementation is immediately built by including a pre-trained ResNet50 backbone as well, without the intermediate step of building a vanilla DeepLab model first.

In this type of architecture, a total of 8 experiments have been conducted. While the main specifications have been tested and experimented with respect to the previous family of models, the different experiments in this case mainly entailed the input size, and the internal parameters of the Atrous Spatial Pyramid Pooling, such as its number of filters and regularization parameters like the different Spatial Dropout rates.

The final version of DeepLab consists of 17 million parameters and 183 total layers, including convolutions, pooling, transposed convolutions, batch normalization and upsampling. Hence it represents a shallower, lighter alternative to the ResNet U-Nets previously discussed.

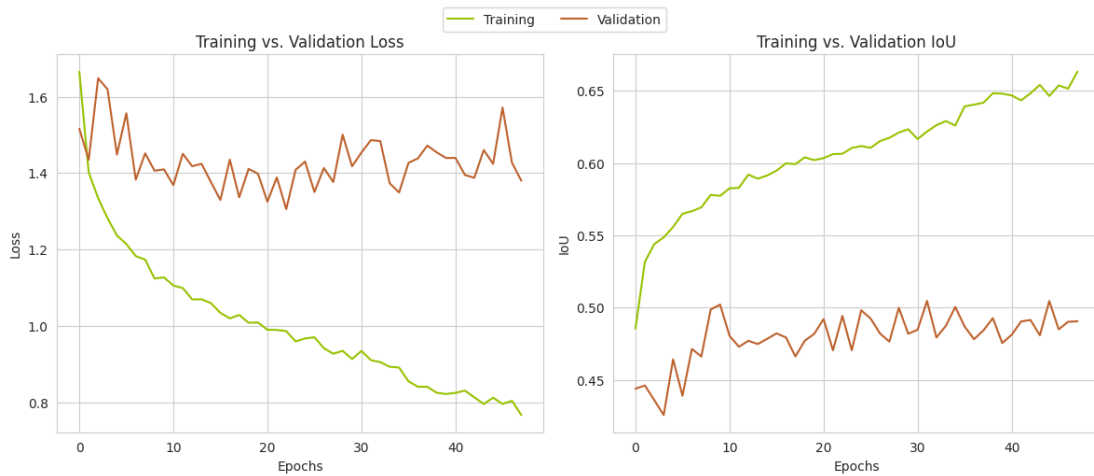


Figure 4.4: DeepLabV3+ Results

Again, the learning curves in Figure 4.4 help understanding and commenting the model performance. The loss curves comparison highlights an already seen pattern: the training loss constantly decreases, meaning that the model is learning from the input images; on the other hand, the validation loss suffers from severe fluctuations, including a spike in the final steps of the training. The minimum value of the validation loss function is achieved around the 20th epoch, with a value of 1.29. After such minimum

is attained, the loss shifts upward, and finally stabilizes around 1.40. Similarly to what encountered in case of the ResNet50, these two movements seem to suggest that the model suffers from overfitting.

The intuitive conclusion provided by the analysis of the loss functions are confirmed in the Mean IoU curves comparison. While the validation Mean IoU initially increases, it subsequently remains just below the level of 0.50. The maximum value is then attained at epoch number 47, with a Mean IoU of 0.51.

Although clear signs of overfitting materialized in this solution as well, this model represents the architecture having the highest validation metric and the lowest minimum loss value across all the proposed solutions.

By looking at the single-class results in Table 4.4, it is evident that, once again, the main disappointing results are associated with the predictions in classes Forest and Barren. The other classes, instead, exhibit solid results as the validation IoU scores is close to their respective training IoU score.

	Background	Building	Road	Water	Barren	Forest	Agriculture	Overall
Validation	0.52	0.54	0.55	0.69	0.27	0.37	0.62	0.51
Training	0.63	0.57	0.64	0.79	0.59	0.72	0.76	0.67

Table 4.4: DeepLabV3+ U-Net IoU scores, single-class and overall, on the validation set.

4.4 Attention U-Net

One important step in the efforts of overcoming the previous models limitations, and hence providing an increasingly better result, concerns the deployment of one U-Net architecture with Attention gates. The potentially beneficial impact of such strategy has been anticipated in section 3.5.

The proposed architecture has the now common ResNet50 backbone at the encoder, and a decoder with four blocks and as many Attention gates working at the skip connections. Following the choice reported in 4.2.1, the backbone’s layers become trainable from 143 onward. The resulting model has a total of 50 million parameters.

The learning curves indicate that, overall, the model achieves decent levels of learning. In fact, the training and validation loss have both a decreasing trend, yet the validation loss in a less decisive fashion. The same pattern is observed with respect to the Mean IoU trajectories, on the right hand panel of Figure 4.5.

However, despite the appreciable property for which this model exhibits a lower degree of overfitting, especially when compared with the other pre-trained models, the general levels of performance are quite lower than expected. This model achieves a top Mean IoU of 0.39 on the validation data, making it the third best performing model, after DeepLabV3+ and ResNet50-U-Net. Such outcome is partially disappointing. Provided that there is no model that can always be the best solution for each heterogeneous application, it is also true that the expectations on this model were higher.

Potential elements that lead to the suboptimal results of the Attention ResNet50 U-Net are the following. By definition, an attention-based model assumes that there exist certain feature maps or images regions that are more important than others. But in case this assumption is not encountered in the real task, then attention-based models can reach poorer results than a standard U-Net, as it could introduce unnecessary complexity. Moreover, attention-based methods require to be trained on very large, extended and well labeled data. If the training and/or the validation data suffer from noisy or inconsistent labeling, then the attention model, as well as any other model, will be misguided, as it will inadvertently amplify noise or focus on irrelevant details.



Figure 4.5: Attention ResNet50 U-Net Results

	Background	Building	Road	Water	Barren	Forest	Agriculture	Overall
Validation	0.49	0.41	0.42	0.61	0.11	0.24	0.46	0.39
Training	0.51	0.37	0.46	0.61	0.10	0.43	0.54	0.43

Table 4.5: ResNet50 U-Net IoU scores, single-class and overall, on the validation set.

5 Results

This chapter provides a general overview over the proposed models and a comparison with the original solutions taken from the paper by Wang et al. [61]. In the second section, an out-of-sample discussion is introduced to check for the external validity of the model.

5.1 Models Comparison

Several critical observations can be drawn from the results exposed in the previous sections. While a simpler model such as the Vanilla U-Net provided a sufficient starting point and was not associated with heavy overfitting behavior, more complex and sophisticated models exploiting pre-trained backbones and ASPP blocks were capable of yielding higher prediction accuracies, at the cost of achieving higher overfitting tendencies. To address this issue, many different changes have been carried out. One, for example, is the L2 Regularizer within the convolutional layers. Moreover, both dropout and spatial dropout have been experimented with different rates and at different extents. Unfortunately, these regularization tools, aimed at reducing the overfitting, were not fully successful in their attempts.

	Background	Building	Road	Water	Barren	Forest	Agriculture	Overall
Baseline U-Net	0.32	0.38	0.41	0.51	0.21	0.28	0.40	0.36
ResNet50 U-Net	0.53	0.53	0.50	0.68	0.29	0.29	0.62	0.49
EfficientNet U-Net	0.48	0.53	0.22	0.06	0.26	0.21	0.41	0.31
DeepLabV3+	0.52	0.54	0.55	0.69	0.27	0.37	0.62	0.51
Attention U-Net	0.49	0.41	0.42	0.61	0.11	0.24	0.46	0.39
Paper HRNet	0.45	0.55	0.57	0.74	0.11	0.45	0.61	0.50
Paper DeepLabV3+	0.43	0.51	0.52	0.74	0.10	0.44	0.59	0.48

Table 5.1: Overall Models Comparison of Validation IoU scores, single-class and overall, on the validation set, including the paper’s results.

Experiments relating to attention-based mechanisms, although theoretically promising, did not provide the expected outcome. These aspects will certainly be deepened in

future research that will follow the current thesis.

Overall, the DeepLabV3+ model with ResNet50 as backbone appeared to be the most successful architecture, with a Mean IoU of 0.51 on the validation data. This model represent a robust solution, that will be considered as a benchmark for future efforts to refine the segmentation accuracy with further, additional models.

When compared to the original paper implementation of DeepLabV3+, the hereby proposed solution of the same model offers a non negligible improvement, by providing an increment of 3 Mean IoU points. Special contribute in achieving this improvement arises from an enhanced accuracy precision with respect to the classes Background and Barren, whereas the original solution outperforms in Forest class, as it might be expected. The DeepLabV3+ model also competes with the original paper’s best proposed result, namely with a HRNet [61]. The two models can be deemed on the same level in terms of prediction quality. However, the fact that this performance was achieved with significantly fewer computational resources highlights the efficiency of the proposed approach. In fact, Wang et al. [61] could rely on a 10.000-epochs-long training session, while in this case the slightly better result is obtained with only a few tens of training steps.

One possible explanation of such result is linked to the larger number of augmentation transformations. While in the benchmark study the augmentation pipeline was limited to a pair of steps, namely random rotation and mirroring, the proposed DeepLabV3+ could rely on more extended augmentation steps, as documented in Section 2.4.7. Additional factors could be related to the use of the Nadam optimizer and a custom loss function tailored for this specific task. All together, these elements likely contributed to a faster, yet reliable prediction quality, capable of competing with state-of-the-art solutions.

A good way to compare models, other than putting side by side their respective evaluation scores, is to analyze and comment a group of representative example predicted masks, identifying strengths and weaknesses of each model.

Figure 5.1 offers some valuable examples, which cover a heterogeneous variety of environments. In the top row of figures, there is one example of very accurate prediction. This image contains every possible class except Forest. DeepLabV3+ confirms to be the best also from a qualitative point of view, as its prediction is more detailed than the other models. The only parts it fails predicting are the small Barren areas, which could have arguably be assigned to agriculture, as there seems no difference between the two areas in the input image. ResNet50 U-Net also proves to be a reliable model, however it is less decisive along the objects’ borders when compared with DeepLab. Other models

are capable of grasping some particular patterns, such as the road or the water areas, however they have significantly lower standards.

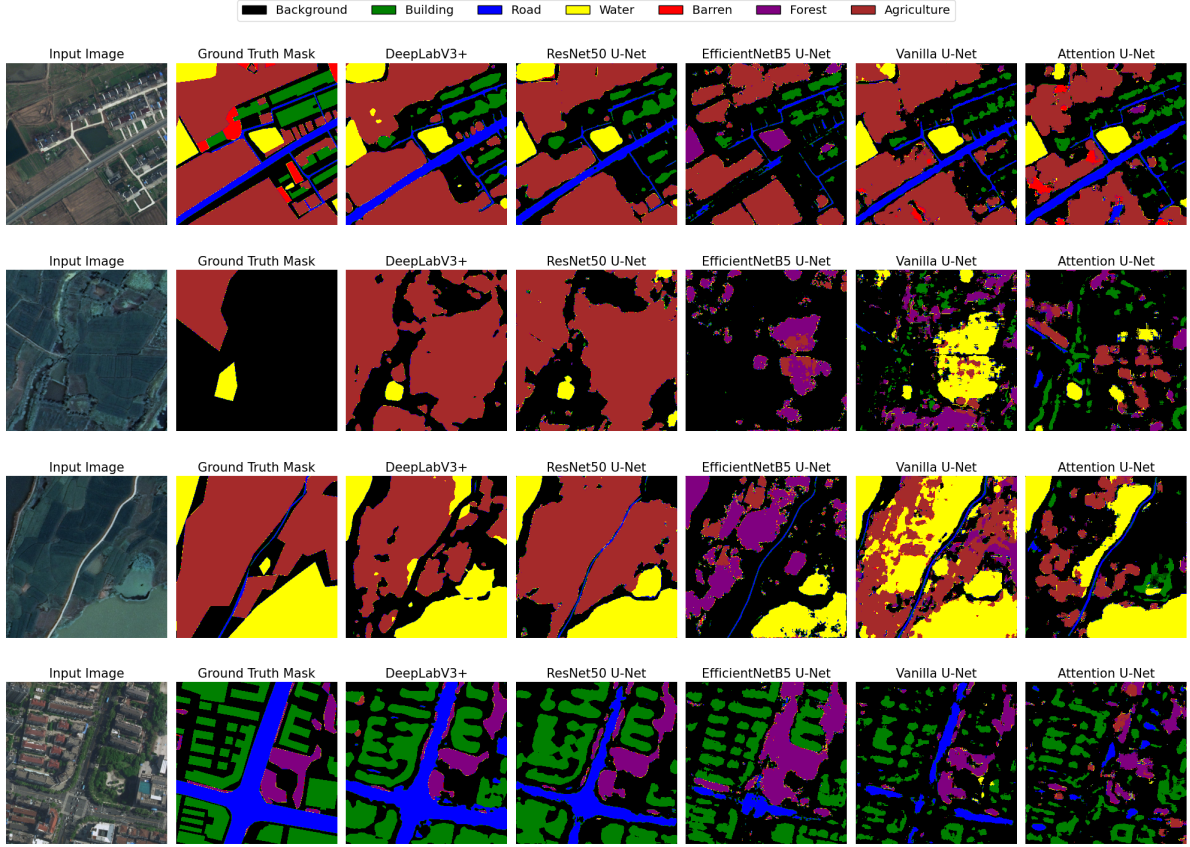


Figure 5.1: From left to right: input images, ground truth masks, and predicted masks for each proposed model.

The second row provides a visual example of a frequent and noteworthy situation encountered in this dataset, noticed during this final assessment phase. That is, the original masks suffer from recurrent, highly inaccurate labeling regarding the pixels of the Forest class. Often, pixels that should belong to the Forest class, are instead labeled as Background. Such inaccurate labeling can negatively impact the learning process of a model, because it ends up associating certain patterns to a certain class, whereas they should be associated with another, leading to confusion and noise. Examples where Forest pixels are incorrectly labeled are very numerous. Background is not the only class that Forest gets wrongly exchanged with: another case is when the Forest label is assigned to Agriculture, and vice versa. Despite the bad labeling, the proposed DeepLab

model does improve upon the original mask: it predicts Forest where there actually is forest in the input image, even when the original pixels are mislabeled. However, it certainly suffers from the inconsistencies and hence it is likely that it would have learned finer relationships, if the data had been provided more precisely.

The third row showcases one limitation of DeepLab. In fact, when the input image contains a thin road, the model struggles in its identification. This pattern has been observed repeatedly in the assessment phase. Other models, instead, seem to provide more robust outcomes related to Road pixels. Moreover, in this image, DeepLab appears to be confused between Water and Agriculture classes in the upper region of the image.

The fourth row moves the focus from rural to urban image example. In this case, differently from instances with thin strips, the model successfully detects and classifies broader road areas. As concerns the Forest areas, DeepLab captures the presence of scattered trees across the urban environment. Building segmentation also shows solid results, considering the difficulty in accurately segmenting smaller objects in the context with other diverse objects. In general, once again DeepLab is more precise with respect to the borders, confirming its higher capability in edge detection.

5.2 Out-of-Sample Predictions

To further assess the generalization ability and the performance of the DeepLab model, a few images have been selected to create an out-of-sample dataset. These images have been taken from Google Earth satellites, in a location close to Parma, Italy. The idea behind this choice is to estimate how the model would perform on unseen data that may differ even significantly from the dataset it has been trained on. However, to ensure consistency, the images have the same physical properties of the dataset, i.e. RGB 1024x1024 and they undergo the same necessary preprocessing steps that allow them to be suitable for the model.

Figure 5.2 includes four examples of such images. Since they are out-of-sample input data, they are not paired with the corresponding ground-truth masks. The first image on the left comprises areas of trees, barren land, and one road. The model is able to produce a remarkably accurate segmentation prediction. The Road is well identified, the Forest areas are correctly predicted as such, and the barren land is well predicted as well. One note relates to the background: on the left, the model assigns the Background class to areas that might belong, in reality, to either Barren or Forest. However, the

decision to assign those regions as Background is reasonable as it is ambiguous, even to the human eye, what those regions actually should belong to.

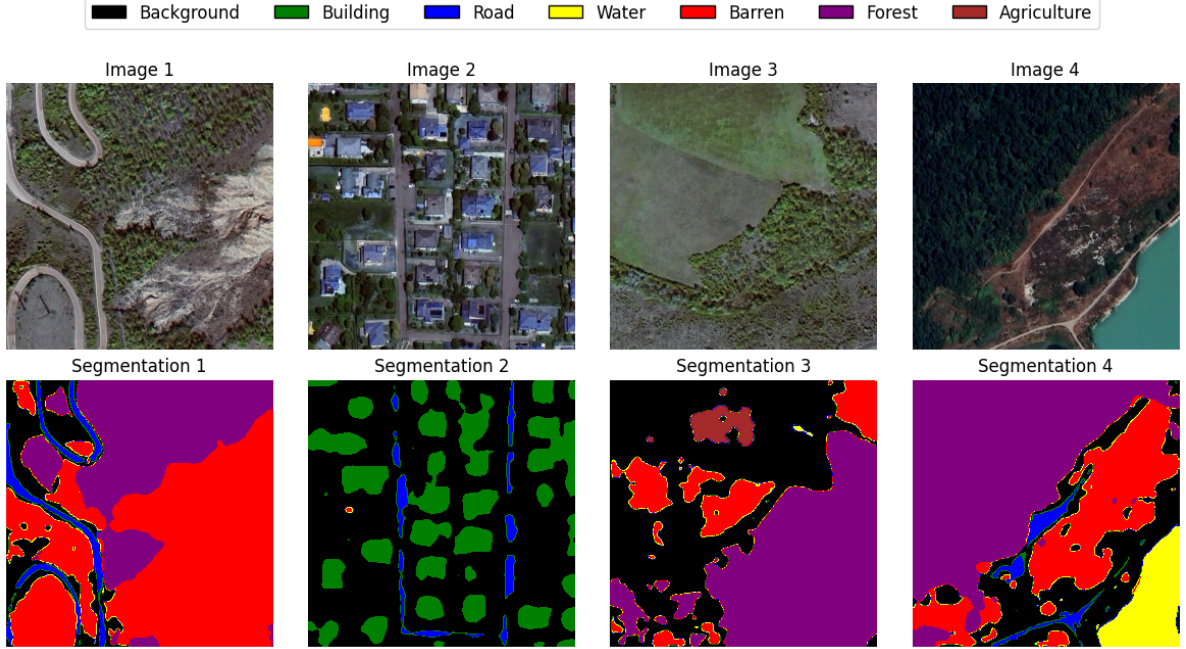


Figure 5.2: Out-of-sample Predictions

The second image captures a different scene. This time a urban area is selected, comprising alleys and houses. In this case, the model is less accurate, more ambiguous in delineating the houses' shapes and in identifying the alleys lines. Despite it struggles in deciding over the precise boundaries of the houses, it does a decent job in recognizing their respective locations. Scattered trees are present, however they are probably not enough to be distinctly detected by the model.

The third image has been selected to illustrate the consequences of the inaccurate annotation phenomenon that has been anticipated in the previous section during the model assessment phase. In the left-hand part of the image, there is an area that is clearly an agricultural field, however the model is uncertain and does not know how to classify it. In fact it produces a fragmented segmentation that assigns a mix of barren, background, and, in a minor part, the right agriculture label. This error may originate from the numerous wrong examples the original training data is subject to.

The fourth image is included as it comprises a water body for completeness, that was missing in the previous three examples. The model behaves very well in recognizing

the lake and its boundaries. It also accurately segments the Forest areas in the top left part of the image. However, more uncertainty is noticed relating to regions containing Barren and a narrow Road. Nonetheless, it successfully captures the broader context of the scene, demonstrating a valuable performance.

Overall, the model demonstrates a satisfying behavior when facing out-of-sample data. It tends to be less accurate with respect to urban areas in general, perhaps due to the small size of the objects and their more intricate features. On the other hand, it performs notably in rural scenes, especially those containing Forest and Barren land, where larger and more distinct features allow for more accurate segmentation.

6 Conclusions

This final chapter summarizes the main insights, findings, and limitation of the implemented models. Furthermore, indications of future work trajectories are provided, as the goal will consist of making the model part of a production tool.

6.1 Main Insights

The motivation of this work is to identify a reliable, well functioning 2D image semantic segmentation model that will be utilized as a tool to enhance the realism of a flight simulator software offered by TXT E-Tech. A variety of models architectures have been carried out as part of this thesis project. To this aim, the study by Wang et al. [61] has been considered as a reference and for the dataset source. These models include simple and lightweight models, such as the Vanilla U-Net, and more sophisticated, complex models such as the Attention U-Net and the DeepLabV3+ with pre-trained backbone. To compare the models, the Mean IoU metric has been selected.

The DeepLabV3+ model demonstrated to yield superior performances when compared to the other competitors presented in Chapter 4. The possible explanation of its success is related to its distinctive element, represented by the Atrous Spatial Pyramid Pooling module, specifically designed to capture multi-scale objects in a given image subject to the segmentation task. Among the other models, the ResNet50 U-Net yielded notable results, whereas the Attention U-Net, which expected to outperform the fully convolutional networks, returned disappointed results.

The DeepLabV3+ model was capable of reaching a Mean IoU of 0.51, surpassing the score of 0.48 achieved by the original study with the same architecture. Potential reasons behind this improvement lie in the more extensive data augmentation transformations, which increase the generalization ability of the model, and in a tailored loss function, which can impact the way the model learns from correct and wrong intermediate predictions. The major limitations of the DeepLabV3+ model emerge when the object to be segmented is of smaller dimension and of less distinct features.

One noteworthy observation regards the quality of the input data. It has been manifold highlighted that, despite the afterward check, several masks suffer from poor, inconsistent labeling, specifically in the regions depicting Forest areas. Such limitation probably plays an important role as obstacle to obtain significant improvements on the overall Mean IoU.

The external validity of the model has been evaluated by performing an out-of-sample analysis. To this aim, the DeepLabV3+ model has been deployed with respect to a few satellite images that have been drawn from Google Earth. The final response of the model can be deemed satisfactory, as it is capable of reproducing the performances seen and discussed with respect to the training LoveDA dataset. On out-of-sample images, the model offers remarkable results when classifying regions of certain classes, whereas it reaffirms its limitations on certain others, as discussed in Section 5.2.

6.2 Future Work

The most important future endeavor will consist of experimenting with different, newer technologies such as the Transformers-based methods. An initial exploration has already been started, and a model prototype will likely be available shortly after the submission of this thesis.

One additional future trajectory is to consider alternative, richer datasets that less suffer from inconsistent labeling with respect to LoveDA. The LoveDA dataset, while useful for benchmarking, has shown issues with inconsistent labeling. Training the implemented models, or even more promising ones like the Transformers, on a more accurate dataset annotations could yield significant improvements and segmentation precision, that would ultimately be beneficial for the flight simulator enhancement thanks to more reliable geographical representations.

Finally, to align better with the specific necessities of the flight simulator, it could be valuable to rearrange the classes labeling distribution. More specifically, merging areas belonging to Barren land with the Agriculture ones may yield practical benefits. This choice could allow to prioritize the identification of classes that have greater relevance for the simulator's functionality.

Bibliography

- [1] Nabila Abraham and Naimul Mefraz Khan. A novel focal tversky loss function with improved attention u-net for lesion segmentation. In *2019 IEEE 16th international symposium on biomedical imaging (ISBI 2019)*, pages 683–687. IEEE, 2019.
- [2] Abien Fred Agarap. Deep learning using rectified linear units (relu), 2019. URL <https://arxiv.org/abs/1803.08375>.
- [3] Salem Saleh Al-Amri, NV Kalyankar, and SD Khamitkar. Image segmentation by using edge detection. *International journal on computer science and engineering*, 2(3):804–807, 2010.
- [4] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 39(12):2481–2495, 2017.
- [5] Pierre Baldi and Peter J Sadowski. Understanding dropout. *Advances in neural information processing systems*, 26, 2013.
- [6] Yoshua Bengio. Practical recommendations for gradient-based training of deep architectures. In *Neural networks: Tricks of the trade: Second edition*, pages 437–478. Springer, 2012.
- [7] Yoshua Bengio, Ian Goodfellow, and Aaron Courville. *Deep learning*, volume 1. MIT press Cambridge, MA, USA, 2017.
- [8] Nils Bjorck, Carla P Gomes, Bart Selman, and Kilian Q Weinberger. Understanding batch normalization. *Advances in neural information processing systems*, 31, 2018.
- [9] Lokesh Borawar and Ravinder Kaur. Resnet: Solving vanishing gradient in deep networks. In *Proceedings of International Conference on Recent Trends in Computing: ICRTC 2022*, pages 235–247. Springer, 2023.

- [10] Alexander Buslaev, Vladimir I Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A Kalinin. Albumentations: fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [11] Guangyong Chen, Pengfei Chen, Yujun Shi, Chang-Yu Hsieh, Benben Liao, and Shengyu Zhang. Rethinking the usage of batch normalization and dropout in the training of deep neural networks. *arXiv preprint arXiv:1905.05928*, 2019.
- [12] Liang-Chieh Chen. Semantic image segmentation with deep convolutional nets and fully connected crfs. *arXiv preprint arXiv:1412.7062*, 2014.
- [13] Liang-Chieh Chen. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017.
- [14] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):834–848, 2017.
- [15] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European conference on computer vision (ECCV)*, pages 801–818, 2018.
- [16] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017.
- [17] Guy Barrett Coleman and Harry C Andrews. Image segmentation by clustering. *Proceedings of the IEEE*, 67(5):773–785, 1979.
- [18] Suresh Dara and Priyanka Tumma. Feature extraction by using deep learning: A survey. In *2018 Second international conference on electronics, communication and aerospace technology (ICECA)*, pages 1795–1801. IEEE, 2018.
- [19] Ilke Demir, Krzysztof Koperski, David Lindenbaum, Guan Pang, Jing Huang, Saikat Basu, Forest Hughes, Devis Tuia, and Ramesh Raskar. Deepglobe 2018: A challenge to parse the earth through satellite images. In *Proceedings of the IEEE*

- conference on computer vision and pattern recognition workshops*, pages 172–181, 2018.
- [20] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [21] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [22] Timothy Dozat. Incorporating nesterov momentum into adam. *no journal*, 2016.
- [23] Christian Garbin, Xingquan Zhu, and Oge Marques. Dropout vs. batch normalization: an empirical study of their impact to deep learning. *Multimedia tools and applications*, 79(19):12777–12815, 2020.
- [24] Alberto Garcia-Garcia, Sergio Orts-Escolano, Sergiu Oprea, Victor Villena-Martinez, and Jose Garcia-Rodriguez. A review on deep learning techniques applied to semantic segmentation. *arXiv preprint arXiv:1704.06857*, 2017.
- [25] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [26] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323. JMLR Workshop and Conference Proceedings, 2011.
- [27] Frauke Günther and Stefan Fritsch. Neuralnet: training of neural networks. *R J.*, 2(1):30, 2010.
- [28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

- [29] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [30] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- [31] Shruti Jadon. A survey of loss functions for semantic segmentation. In *2020 IEEE conference on computational intelligence in bioinformatics and computational biology (CIBCB)*, pages 1–7. IEEE, 2020.
- [32] Hassana Grema Kaganami and Zou Beiji. Region-based segmentation versus edge detection. In *2009 Fifth International Conference on Intelligent Information Hiding and Multimedia Signal Processing*, pages 1217–1221. IEEE, 2009.
- [33] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [34] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [35] Siddharth Krishna Kumar. On weight initialization in deep neural networks. *arXiv preprint arXiv:1704.08863*, 2017.
- [36] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521 (7553):436–444, 2015.
- [37] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems*, 33(12):6999–7019, 2021.
- [38] Xiaolong Liu, Zhidong Deng, and Yuhan Yang. Recent progress in semantic image segmentation. *Artificial Intelligence Review*, 52:1089–1106, 2019.
- [39] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.

- [40] Zifei Luo, Wenzhu Yang, Yunfeng Yuan, Ruru Gou, and Xiaonan Li. Semantic segmentation of agricultural images: A survey. *Information Processing in Agriculture*, 2023.
- [41] Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. Rectifier nonlinearities improve neural network acoustic models. In *Proc. icml*, volume 30, page 3. Atlanta, GA, 2013.
- [42] Zhaoyang Niu, Guoqiang Zhong, and Hui Yu. A review on the attention mechanism of deep learning. *Neurocomputing*, 452:48–62, 2021.
- [43] Ozan Oktay, Jo Schlemper, Loic Le Folgoc, Matthew Lee, Mattias Heinrich, Kazunari Misawa, Kensaku Mori, Steven McDonagh, Nils Y Hammerla, Bernhard Kainz, et al. Attention u-net: Learning where to look for the pancreas. *arXiv preprint arXiv:1804.03999*, 2018.
- [44] K O’Shea. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [45] Niall O’Mahony, Sean Campbell, Anderson Carvalho, Suman Harapanahalli, Gustavo Velasco Hernandez, Lenka Krpalkova, Daniel Riordan, and Joseph Walsh. Deep learning vs. traditional computer vision. In *Advances in Computer Vision: Proceedings of the 2019 Computer Vision Conference (CVC), Volume 1 1*, pages 128–144. Springer, 2020.
- [46] Md Atiqur Rahman and Yang Wang. Optimizing intersection-over-union in deep neural networks for image segmentation. In *International symposium on visual computing*, pages 234–244. Springer, 2016.
- [47] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.
- [48] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.

- [49] T-YLPG Ross and GKHP Dollár. Focal loss for dense object detection. In *proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2980–2988, 2017.
- [50] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization? *Advances in neural information processing systems*, 31, 2018.
- [51] Mehmet Sezgin and Buğlent Sankur. Survey over image thresholding techniques and quantitative performance evaluation. *Journal of Electronic imaging*, 13(1): 146–168, 2004.
- [52] Sagar Sharma, Simone Sharma, and Anidhya Athaiya. Activation functions in neural networks. *Towards Data Sci*, 6(12):310–316, 2017.
- [53] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019.
- [54] Nahian Siddique, Sidike Paheding, Colin P Elkin, and Vijay Devabhaktuni. U-net and its variants for medical image segmentation: A review of theory and applications. *IEEE access*, 9:82031–82057, 2021.
- [55] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [56] Charles Sutton, Andrew McCallum, et al. An introduction to conditional random fields. *Foundations and Trends® in Machine Learning*, 4(4):267–373, 2012.
- [57] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.
- [58] Jonathan Tompson, Ross Goroshin, Arjun Jain, Yann LeCun, and Christoph Breger. Efficient object localization using convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 648–656, 2015.

- [59] Xin-Yi Tong, Gui-Song Xia, Qikai Lu, Huanfeng Shen, Shengyang Li, Shucheng You, and Liangpei Zhang. Land-cover classification with high-resolution remote sensing images using transferable deep models. *Remote Sensing of Environment*, 237:111322, 2020.
- [60] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [61] Junjue Wang, Zhuo Zheng, Ailong Ma, Xiaoyan Lu, and Yanfei Zhong. Loveda: A remote sensing land-cover dataset for domain adaptive semantic segmentation. *arXiv preprint arXiv:2110.08733*, 2021.
- [62] Zhaobin Wang, E Wang, and Ying Zhu. Image segmentation evaluation: a survey of methods. *Artificial Intelligence Review*, 53(8):5637–5674, 2020.
- [63] Yutong Xie, Bing Yang, Qingbiao Guan, Jianpeng Zhang, Qi Wu, and Yong Xia. Attention mechanisms in medical image segmentation: A survey. *arXiv preprint arXiv:2305.17937*, 2023.
- [64] Kaichao You, Mingsheng Long, Jianmin Wang, and Michael I Jordan. How does learning rate decay help modern neural networks? *arXiv preprint arXiv:1908.01878*, 2019.
- [65] F Yu. Multi-scale context aggregation by dilated convolutions. *arXiv preprint arXiv:1511.07122*, 2015.
- [66] Afia Zafar, Muhammad Aamir, Nazri Mohd Nawi, Ali Arshad, Saman Riaz, Abdulrahman Alruban, Ashit Kumar Dutta, and Sultan Almotairi. A comparison of pooling methods for convolutional neural networks. *Applied Sciences*, 12:8643, 2022.